

Conversación ChatGPT

<https://chatgpt.com/share/6ac262d4-3d1c-83e8-9e55-c12e25ef4274>

Tú #1

<<https://tenancy.dev/docs/tenancy/2.x/tenant-what-is>>Para que es esta librería

ChatGPT #2

Sí. Esa librería es stancl/tenancy para Laravel y sirve principalmente para construir aplicaciones multi-tenant (multicliente).

En términos simples:

Una sola aplicación Laravel puede atender a muchas empresas/clientes, manteniendo sus datos separados.

La documentación que pasaste corresponde a Tenancy 2.x; actualmente existe una versión más reciente, por lo que no conviene copiar esa documentación directamente a un proyecto nuevo sin revisar compatibilidad. Tenancy for Laravel+1

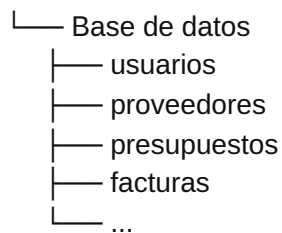
Ejemplo práctico

Imagina que tienes GestiónPlus.

Sin multi-tenancy:

``

GestiónPlus



,

Si GestiónPlus tiene:

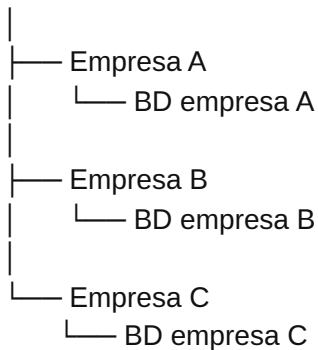
- * Empresa A
- * Empresa B
- * Empresa C

tienes que asegurarte manualmente de que:

Empresa A → solamente vea sus datos
Empresa B → solamente vea sus datos
Empresa C → solamente vea sus datos

Con stanci/tenancy puedes tener algo como:

app.gestionplus.com.mx



Incluso puedes identificar al cliente mediante dominio/subdominio:

empresa-a.gestionplus.com.mx
empresa-b.gestionplus.com.mx
empresa-c.gestionplus.com.mx

El paquete detecta el tenant y puede cambiar automáticamente el contexto de la aplicación, incluyendo la conexión a la base de datos. DeepWiki+1

¿Qué significa "Tenant"?

Un tenant es básicamente un cliente/empresa independiente dentro de tu aplicación.

Por ejemplo:

```
Tenant::create([  
  'plan' => 'premium',  
]);
```

Ese tenant puede tener:

、
ID: empresa-a
Nombre: Constructora ABC
Dominio: abc.gestionplus.com.mx
Plan: premium

、
Y cuando entra una petición a:

、
<https://abc.gestionplus.com.mx>

、
Laravel puede saber:

、
Tenant actual = Constructora ABC

、
y trabajar con los datos correspondientes. Tenancy for Laravel

Hay dos formas principales

1\ Una base de datos por empresa

、
gestionplus_central
gestionplus_empresa_a
gestionplus_empresa_b
gestionplus_empresa_c

、
Es una de las arquitecturas principales soportadas por el paquete y proporciona un aislamiento fuerte entre clientes. DeepWiki

2\ Una sola base de datos

、
users

id | tenant_id | name
1 | empresa_a | Juan
2 | empresa_b | Pedro
3 | empresa_a | Luis

Aquí todos comparten la BD, pero los registros se separan mediante `tenant_id`. También es posible con el ecosistema de Tenancy, aunque requiere plantear correctamente el aislamiento de consultas. DeepWiki

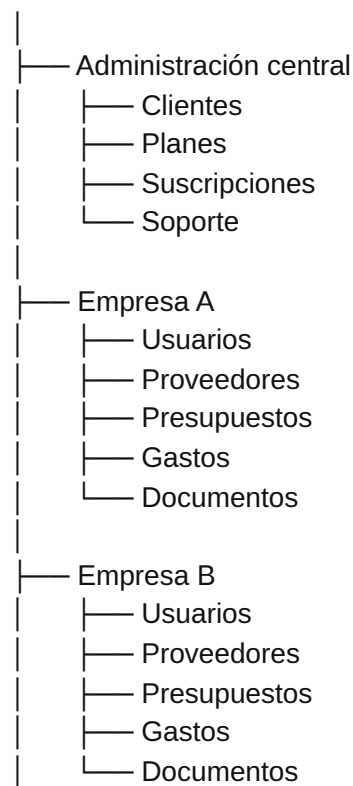
¿Para qué te serviría a ti?

Para un sistema como GestiónPlus, tiene bastante sentido si la intención es convertirlo realmente en un SaaS.

Por ejemplo:

,

GestiónPlus



,

Y el código Laravel puede seguir siendo prácticamente el mismo para todos los clientes; el paquete se encarga de inicializar el contexto del tenant y, dependiendo de la arquitectura, cambiar conexiones y otros recursos. Packagist

Pero hay una distinción importante: si actualmente GestiónPlus es para una sola empresa y simplemente quieres separar módulos, usuarios o permisos, no necesitas Tenancy. Para eso bastan autenticación, roles/permisos y relaciones normales de Laravel.

Si lo que quieres es que GestiónPlus pueda tener 10, 100 o 1,000 empresas diferentes usando la misma instalación, entonces sí es justamente el tipo de problema que `stanc/tenancy` pretende resolver.

Tú #3

GestiónPlus | ├── Administración central | ├── Clientes | ├── Planes | ├──
Suscripciones | ├── Soporte | ├── Empresa A | ├── Usuarios | ├── Proveedores | ├──
Presupuestos | ├── Gastos | ├── Documentos | ├── Empresa B | ├── Usuarios | ├──
Proveedores | ├── Presupuestos | ├── Gastos | ├── Documentos Me resulta curioso.
Aver dime cómo interpretas Gestiónplus como app

ChatGPT #4

Por lo que hemos hablado de GestiónPlus, yo la interpreto no como una simple app de proveedores, sino como una plataforma administrativa/operativa que está empezando a tomar forma de SaaS empresarial.

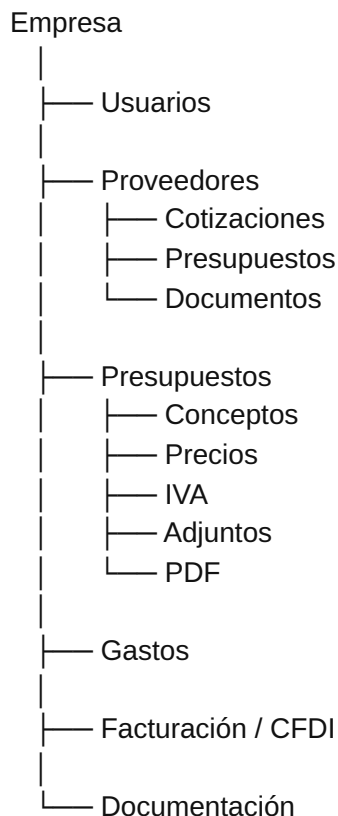
Mi lectura sería esta:

1\ El núcleo actual

GestiónPlus parece girar alrededor de la gestión de operaciones de una empresa, especialmente empresas que manejan compras, proveedores, presupuestos y documentación.

El flujo conceptual que veo es:

..



Y Construcc lo interpreto más como una aplicación complementaria/móvil/PWA que consume parte de este ecosistema.

2\ Pero hay algo más interesante

Por las cosas que has ido desarrollando, veo que GestiónPlus tiene potencial para convertirse en algo parecido a:

Un ERP ligero orientado inicialmente a empresas de construcción y servicios, pero con arquitectura suficientemente general para otros negocios.

Esto encaja especialmente con lo que comentaste sobre los presupuestos.

No lo veo como:

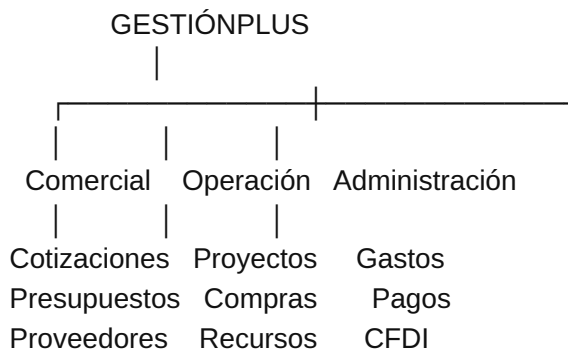
,

"software para hacer cotizaciones"

,

sino como:

,



,

Y construcción sería el primer mercado, no necesariamente la definición completa del producto.

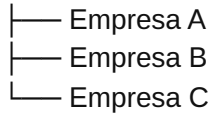
3\ Aquí es donde entra Tenancy

Si esa interpretación es correcta, entonces el modelo que te puse anteriormente adquiere bastante sentido.

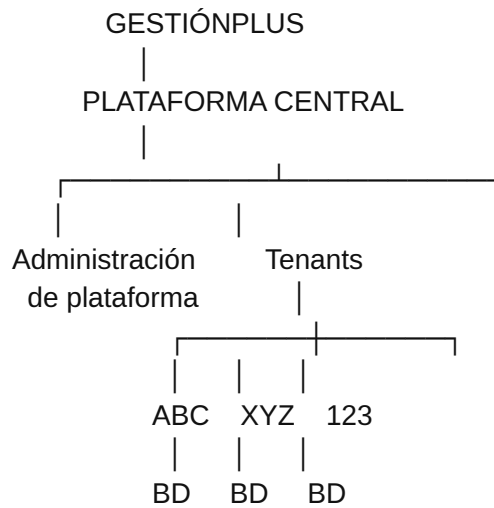
Pero yo no lo diseñaría exactamente como:

,

GestiónPlus



Yo lo conceptualizaría así:

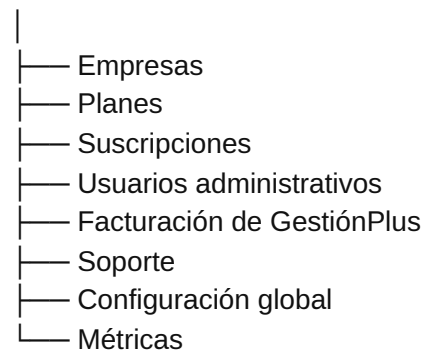


La administración central no pertenece a ninguna empresa.

Es el nivel de GestiónPlus.

Ahí tendrías cosas como:

Platform



Mientras que cada empresa tendría su propio universo:

Tenant: Constructora ABC

- |
- Usuarios
- Roles
- Proveedores
- Clientes
- Presupuestos
- Gastos
- Compras
- Documentos
- Proyectos
- Configuración

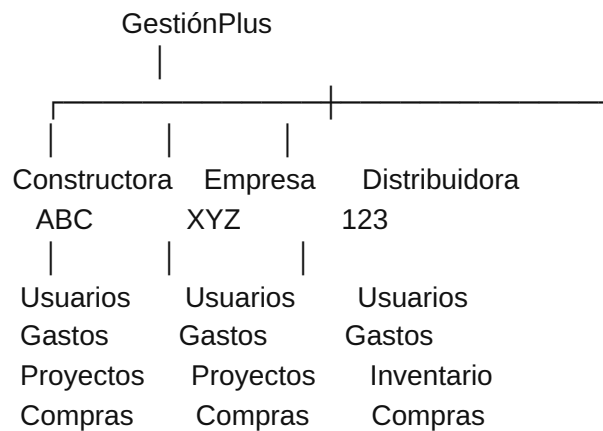
,

4\ Y aquí veo una oportunidad importante

La arquitectura te permitiría que GestiónPlus deje de estar conceptualmente ligado a SJS.

Por ejemplo:

,



,

Todos utilizan la misma aplicación.

Pero cada uno tiene su propio contexto.

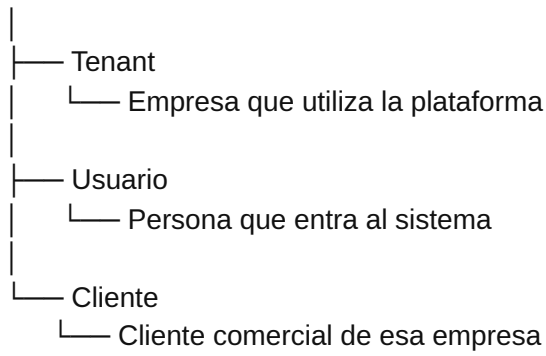
Eso es precisamente donde stancel/tenancy empieza a ser interesante.

5\ Incluso cambiaría un poco tu concepto de "Cliente"

Hay tres conceptos diferentes que conviene no mezclar:

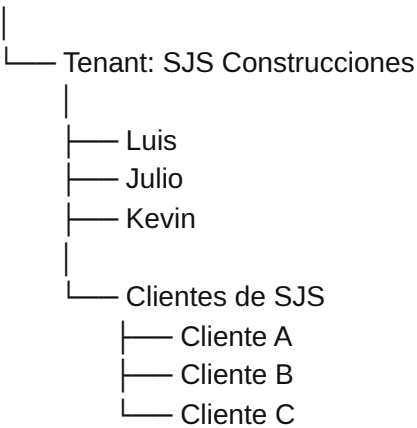
,

GestiónPlus



Por ejemplo:

GestiónPlus



Ese detalle es muy importante.

Porque si utilizas clientes para referirte tanto a los clientes de GestiónPlus como a los clientes de una empresa usuaria, tarde o temprano tendrás una ensalada de tablas bastante creativa.

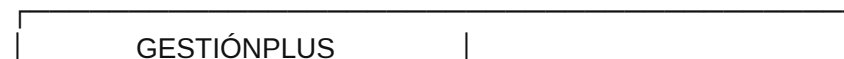
6\ Mi interpretación actual de GestiónPlus

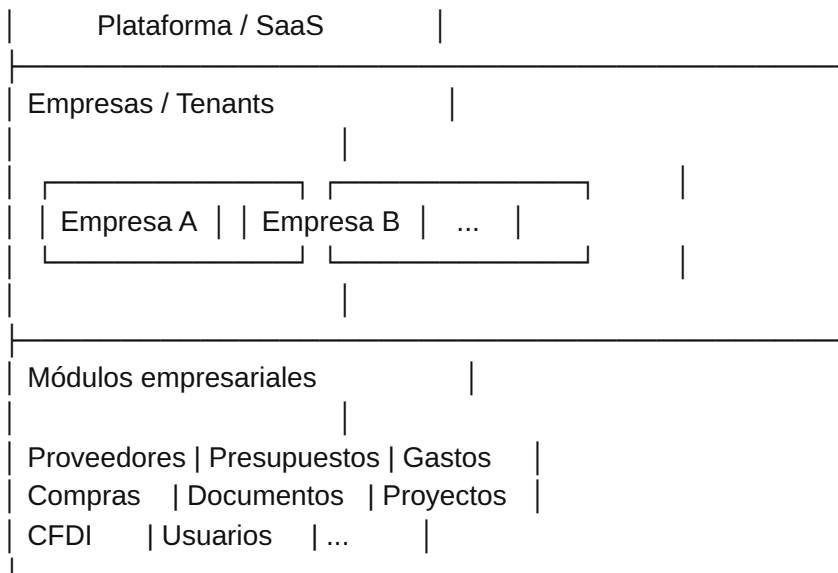
Si tuviera que definirlo en una frase:

GestiónPlus es una plataforma SaaS de gestión empresarial, inicialmente orientada a construcción, que centraliza proveedores, presupuestos, compras, gastos, documentación y procesos administrativos.

Y veo tres niveles:

,





,

Y Construcc podría ser una interfaz especializada dentro del ecosistema, no necesariamente "otro producto completamente separado".

Lo que me parece especialmente interesante

Por lo que ya tienes construido, no empezaría introduciendo Tenancy mañana mismo.

Primero definiría correctamente el modelo de negocio y dominio de GestiónPlus:

,

Platform

↓

Tenant / Empresa

↓

Usuarios + Roles

↓

Módulos

↓

Datos de la empresa

,

Después decidiría si conviene:

* BD única + tenant_id

* BD independiente por tenant

* o una arquitectura híbrida.

Para GestiónPlus, BD independiente por empresa puede ser bastante atractiva si realmente quieres venderlo como SaaS, especialmente por aislamiento de datos, respaldos y

eventualmente migración de clientes.

Y ahí sí stancl/tenancy` deja de ser una librería curiosa y pasa a resolver una parte bastante seria de la arquitectura.

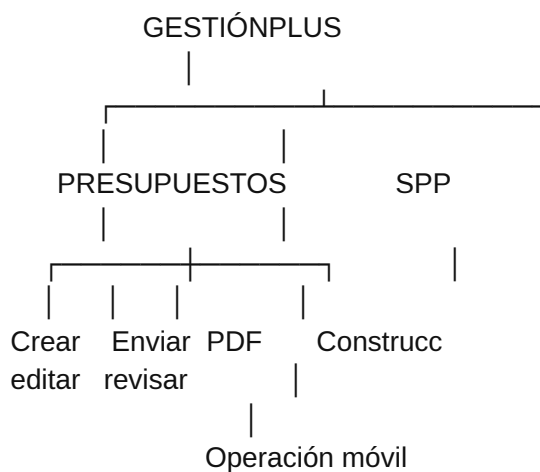
Tú #5

Aver actualmente la app solo tiene las funciones de presupuestos y SPP con construcc

ChatGPT #6

Correcto. Entonces mi interpretación anterior estaba adelantando demasiado la arquitectura. Si actualmente GestiónPlus solamente tiene Presupuestos y su integración/relación con SPP mediante Construcc, yo la veo así:

..



,

Entonces, ¿qué es GestiónPlus hoy?

Yo la definiría como:

Una plataforma de gestión de presupuestos y procesos relacionados, actualmente enfocada en el flujo de proveedores y conectada con Construcc/SPP.

No la llamaría todavía ERP.

Tampoco diría que ya es un SaaS multiempresa.

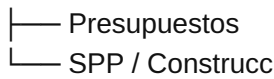
Eso sería hacia dónde podría evolucionar, no lo que actualmente es.

Y aquí cambia bastante la cuestión de Tenancy

Si actualmente tienes:

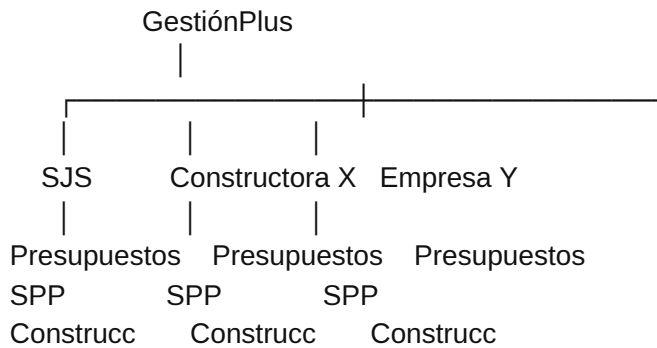
,

GestiónPlus
└─ SJS



meter stancl/tenancy solo porque "algún día podría haber empresas A, B y C" probablemente sería prematuro.

Pero si el objetivo del proyecto es que GestiónPlus eventualmente sea:



entonces sí conviene pensar en tenancy desde el diseño, antes de que tengas 30 módulos y 200 tablas.

La pregunta importante ya no es "¿para qué sirve Tenancy?", sino:

¿GestiónPlus está pensado para convertirse en un producto que pueda ser utilizado por varias empresas independientes, o seguirá siendo el sistema de SJS?

Porque esa respuesta cambia bastante la arquitectura que te recomendaría. tenancy.dev

Tú #7

Está pensando para muchas empresas con la misma ba peor deja dime el concepto yo no siento la app concretada

ChatGPT #8

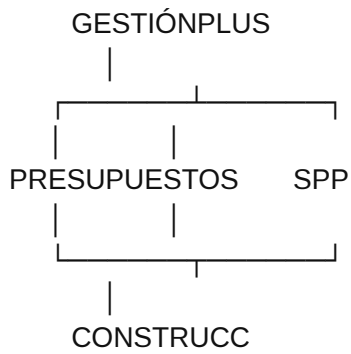
Sí. Y con esa aclaración, creo que entiendo mejor lo que estás sintiendo: GestiónPlus todavía no tiene una identidad de producto completamente cerrada.

Ahora mismo tienes funciones reales, pero eso no necesariamente significa que exista un concepto de producto detrás de ellas.

Yo separaría 3 cosas

1\ Lo que GestiónPlus es actualmente

..



Es decir:

GestiónPlus actualmente es el sistema que administra el proceso de presupuestación, mientras que Construcc/SPP participa en la operación de ese proceso.

Pero aquí aparece el problema que probablemente estás percibiendo:
¿Qué problema general resuelve GestiónPlus?

Porque "hacer presupuestos" es una función, no necesariamente el concepto del producto.

Yo veo un concepto más interesante

Por lo que has construido, yo no plantearía GestiónPlus como:
"Una aplicación para hacer presupuestos."

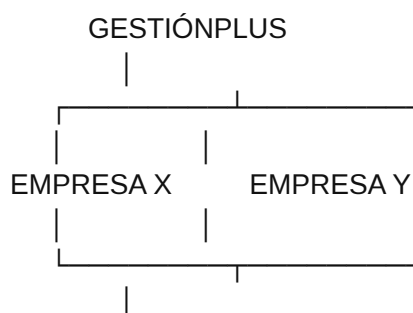
Eso la hace demasiado pequeña.

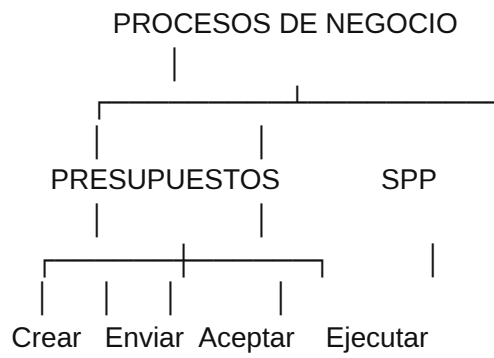
Tampoco la vendería todavía como ERP.

Yo la conceptualizaría como:
Una plataforma para gestionar procesos comerciales y operativos de una empresa, comenzando por el presupuesto como punto de entrada.

El presupuesto sería el núcleo inicial, no el producto completo.

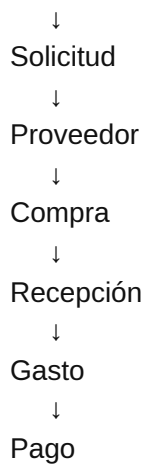
Algo así:





Y posteriormente podrían aparecer:

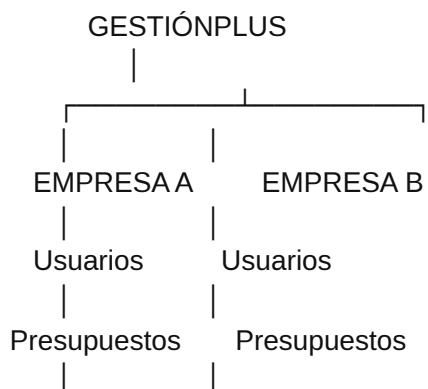
Presupuesto

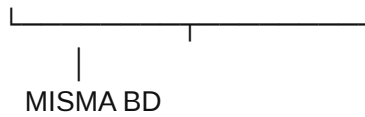


Ahí empieza a aparecer un producto, no solamente un conjunto de módulos.

Y lo de "muchas empresas con la misma BD" es importante

Entonces tu modelo conceptual sería:





,

Por ejemplo:

,

presupuestos

id	tenant_id	folio	total
1	10	P-001	150,000
2	10	P-002	80,000
3	25	P-001	230,000

,

La empresa 10 jamás debería poder ver el registro tenant_id = 25.

Ahí sí aparece una razón concreta para utilizar un enfoque multi-tenant. Tenancy está precisamente orientado a identificar el tenant actual y aislar el contexto de cada empresa.

Tenancy

Pero creo que tu problema es otro

Creo que cuando dices:

"No siento la app concretada"

no estás hablando realmente de programación.

Estás hablando de producto.

Tienes:

- * Presupuestos
- * SPP
- * Construcc
- * proveedores
- * PDFs
- * estados
- * usuarios
- * etc.

Pero posiblemente todavía no puedes decir:

"GestiónPlus sirve para ______."

sin empezar a enumerar módulos.

Y eso es una señal bastante buena de que primero hay que definir el concepto antes de seguir agregando funcionalidades.

Yo empezaría por esta pregunta

No:

"¿Qué módulo le agregamos a GestiónPlus?"

Sino:

"¿Cuál es el proceso empresarial que queremos que GestiónPlus controle de principio a fin?"

Porque si encontramos ese proceso, los módulos salen casi solos.

Por ejemplo, si descubrimos que el verdadero concepto es:

"Gestionar desde que una empresa necesita algo hasta que lo compra, lo recibe y lo paga."

entonces:

、

Necesidad

↓

Solicitud

↓

Presupuesto

↓

Comparación

↓

Aprobación

↓

Proveedor

↓

Compra

↓

Recepción

↓

Gasto

↓

Pago

..

Y de repente Presupuestos y SPP dejan de parecer dos cosas aisladas: se convierten en partes de un flujo.

Eso, en mi opinión, es lo que actualmente le falta definir a GestiónPlus: no más funciones, sino su columna vertebral.

Tú #9

Esta nexprov que es la app de catálogos empresariales y se implementó importaciones de datos de productos y cotizaciones comparte la API de Gestiónplus.

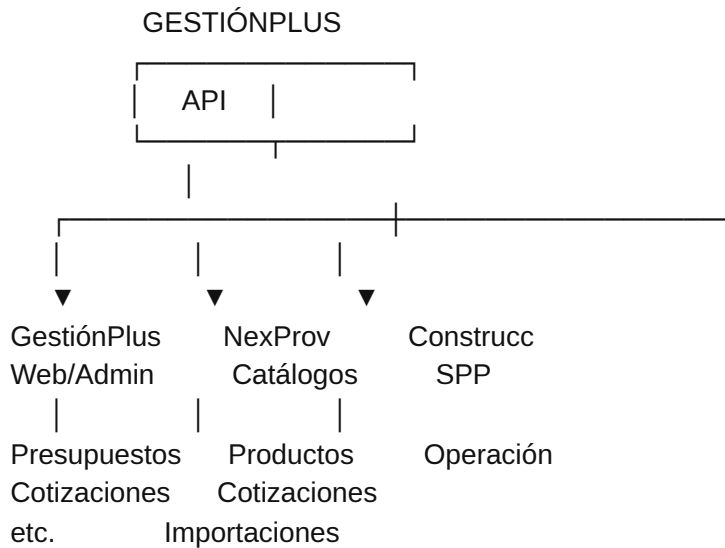
ChatGPT #10

Sí, ese dato cambia bastante la interpretación. NexProv es una pieza importante para entender qué es realmente GestiónPlus.

Por lo que describes, yo ya no vería GestiónPlus como "la app de presupuestos". Lo interpretaría como una plataforma central de información y procesos comerciales, donde distintas aplicaciones consumen el mismo backend.

El ecosistema empieza a verse así

..



,

Y eso es mucho más interesante.

NexProv me da una pista del concepto

Si NexProv:

- * administra catálogos empresariales,
- * importa productos,
- * importa cotizaciones,
- * comparte la API de GestiónPlus,

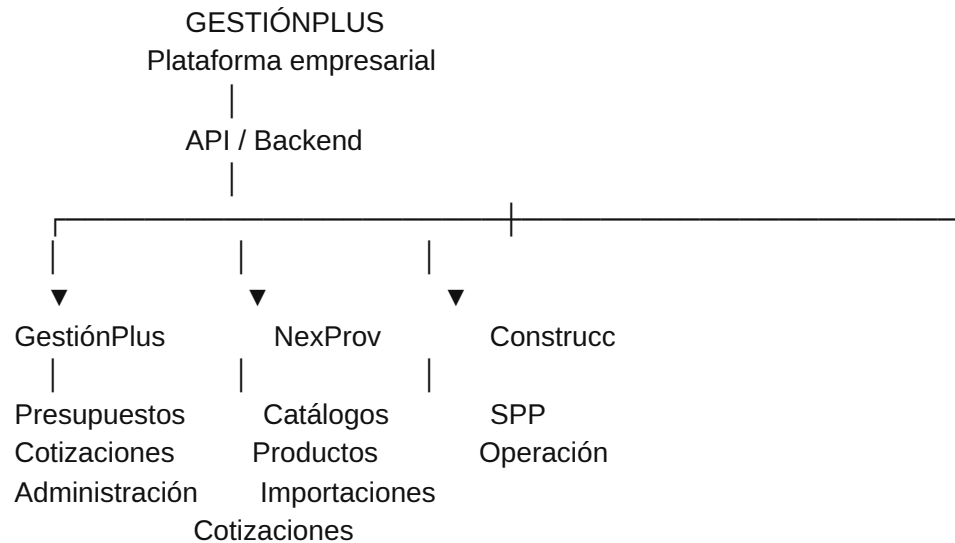
entonces la API ya no debería pensarse simplemente como:
"API de la aplicación de presupuestos."

La API empieza a ser:
la capa de negocio central del ecosistema GestiónPlus.

Las aplicaciones son clientes de esa plataforma.

Yo lo conceptualizaría así

,



,

Esto explica algo que probablemente estabas percibiendo: la aplicación que llamas "GestiónPlus" es solamente una de las interfaces del sistema.

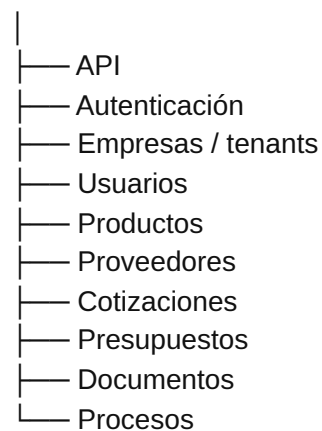
Y aquí aparece una distinción importante

Yo separaría:

GestiónPlus como plataforma

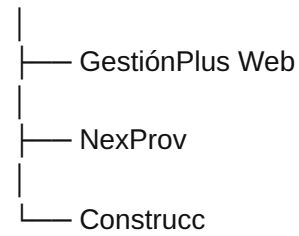
,

GestiónPlus



Aplicaciones

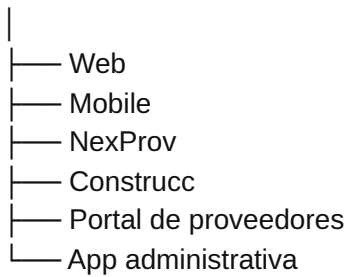
Apps



Esto es bastante más sólido arquitectónicamente.

Por ejemplo, mañana podrías crear:

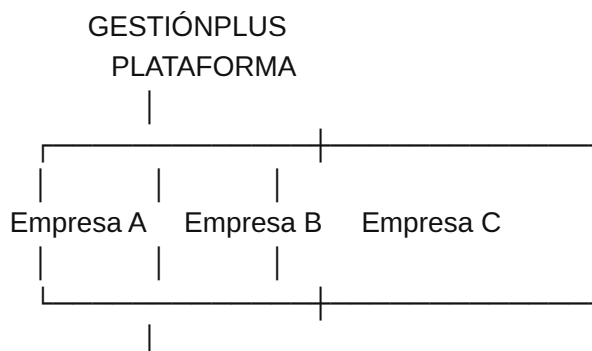
GestiónPlus API

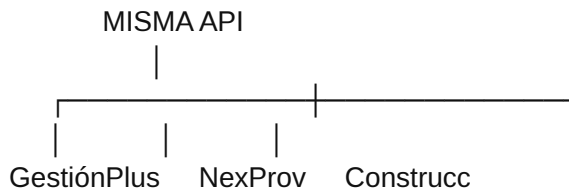


Todas consumen el mismo dominio de negocio.

Y ahora sí entiendo por qué te llamó la atención Tenancy

Si quieres que muchas empresas utilicen el mismo sistema y la misma API, entonces el concepto podría ser:





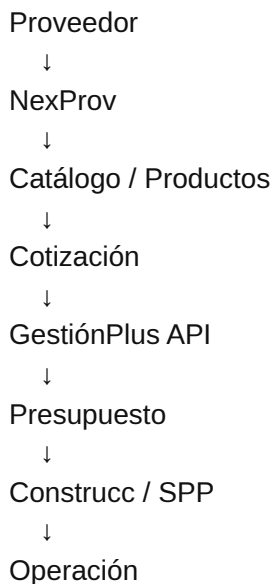
,

Y cada aplicación sabe para qué parte del proceso existe, pero ninguna necesita convertirse en "el sistema completo".

Incluso NexProv puede tener un papel estratégico

Podría existir un flujo como:

,



,

Eso empieza a parecer un ecosistema de gestión empresarial, no tres aplicaciones independientes.

Y ahí sí yo tendría mucho cuidado con meter stanc/tenancy` sin antes definir el dominio.

Porque la pregunta arquitectónica correcta sería:

¿Qué entidades pertenecen a GestiónPlus como plataforma y cuáles pertenecen a cada empresa (tenant)?

Por ejemplo, un producto de un catálogo público podría ser global, mientras que el precio negociado, proveedor, presupuesto y documentos podrían pertenecer a una empresa concreta.

Ese tipo de decisión es precisamente la que determinaría si Tenancy encaja y cómo debería

implementarse.

Si quieres, podemos hacer algo más útil: mapear NexProv + GestiónPlus + Construcc + SPP como un solo ecosistema, identificar qué hace realmente cada uno y, a partir de ahí, definir cuál debería ser el concepto de GestiónPlus.

Tú #11

App.gestionplus.com.mx app.nexprov.com.mx

ChatGPT #12

Exacto. Eso refuerza bastante la separación que estás haciendo:

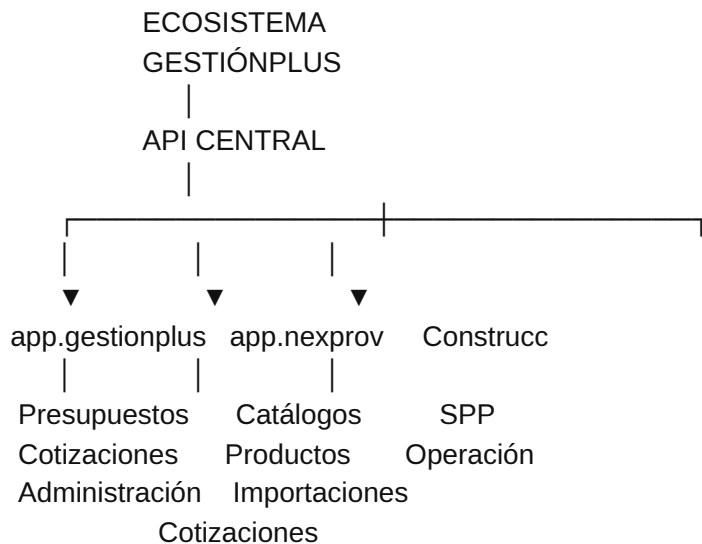
* app.gestionplus.com.mx → GestiónPlus

* app.nexprov.com.mx → NexProv

Ambas son aplicaciones independientes a nivel de frontend/dominio, pero comparten la API de GestiónPlus. GestionPlus+1

Yo empezaría a dibujar el concepto así:

..



,

Y esto cambia una cosa importante:

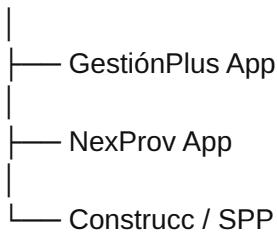
GestiónPlus probablemente no debería entenderse como "una app"

Sino como la plataforma/backend que sostiene un conjunto de aplicaciones empresariales.

,

GESTIÓNPLUS = plataforma





..

Eso también explica por qué puedes sentir que GestiónPlus no está "concretada": estás intentando encontrarle identidad a una aplicación cuando posiblemente el producto real es el ecosistema completo.

Y aquí hay una pregunta que considero fundamental antes de tocar Tenancy:
¿NexProv y GestiónPlus están pensadas para que una misma empresa pueda usar ambas, compartiendo proveedores/productos/cotizaciones/presupuestos, o NexProv es un producto independiente que simplemente reutiliza infraestructura de GestiónPlus?

Esa diferencia nos diría muchísimo sobre cuál debería ser el verdadero concepto de GestiónPlus.

Tú #13

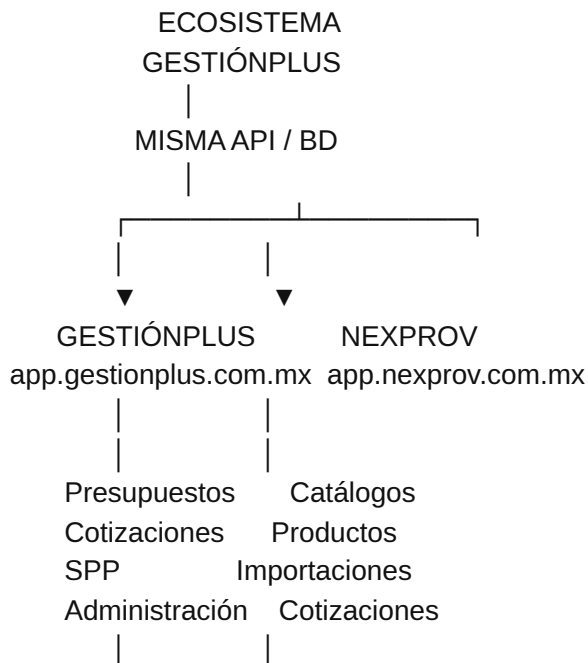
Son gemelos

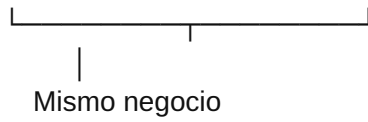
ChatGPT #14

Sí. "Gemelos" es una descripción mucho más acertada que decir que NexProv es simplemente otra app que consume la API.

Yo lo interpretaría así:

..





,

¿Qué significa "gemelos"?

No significa que sean idénticos.

Significa que representan dos caras del mismo sistema.

Por ejemplo:

NexProv está más cerca de:

"¿Qué puedo comprar y a qué precio?"

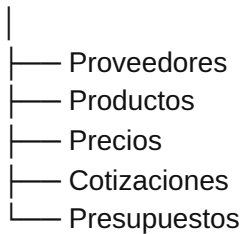
Mientras que GestiónPlus está más cerca de:

"¿Qué necesito presupuestar y cómo gestiono esa operación?"

Y ambas necesitan la misma información:

,

Empresa



,

Por eso tiene sentido que compartan API.

Entonces yo ya no llamaría a GestiónPlus "una app de presupuestos"

Lo que empieza a emerger es algo como:

GestiónPlus/NexProv: un ecosistema empresarial para conectar la oferta de proveedores con los procesos de presupuestación y operación de las empresas.

Y esto tiene una consecuencia arquitectónica importante:

La empresa es probablemente el centro.

No:

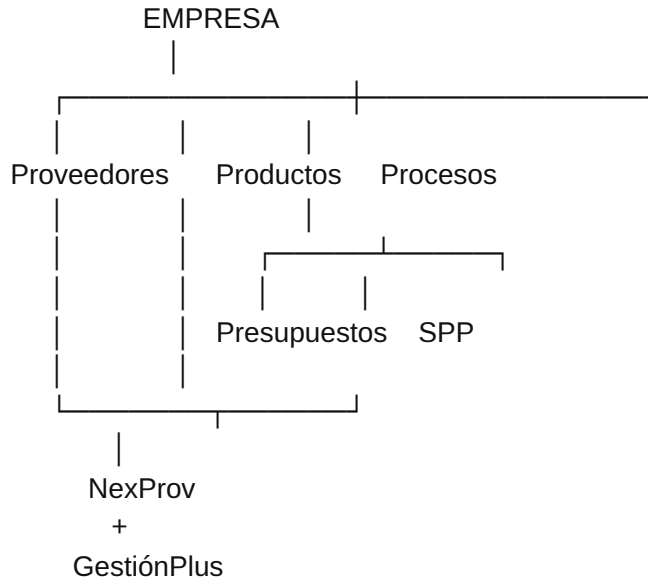
,

Presupuesto
└─ Empresa

,

sino:

,



,

Y ahí es donde la idea de multi-tenancy empieza a tener mucho sentido.

Cada empresa tendría su propio contexto:

,

Tenant A

- ─ usuarios
- ─ proveedores
- ─ productos
- ─ cotizaciones
- ─ presupuestos
- ─ SPP

Tenant B

- ─ usuarios
- ─ proveedores
- ─ productos
- ─ cotizaciones
- ─ presupuestos
- ─ SPP

,

Pero las aplicaciones:

app.gestionplus.com.mx
app.nexprov.com.mx

son simplemente dos interfaces del mismo ecosistema.

Eso, para mí, empieza a darle a GestiónPlus una identidad bastante más concreta. El siguiente paso sería definir qué es el núcleo común de los gemelos y qué pertenece exclusivamente a NexProv o GestiónPlus. Ahí es donde se puede diseñar la arquitectura sin inventar módulos porque sí.

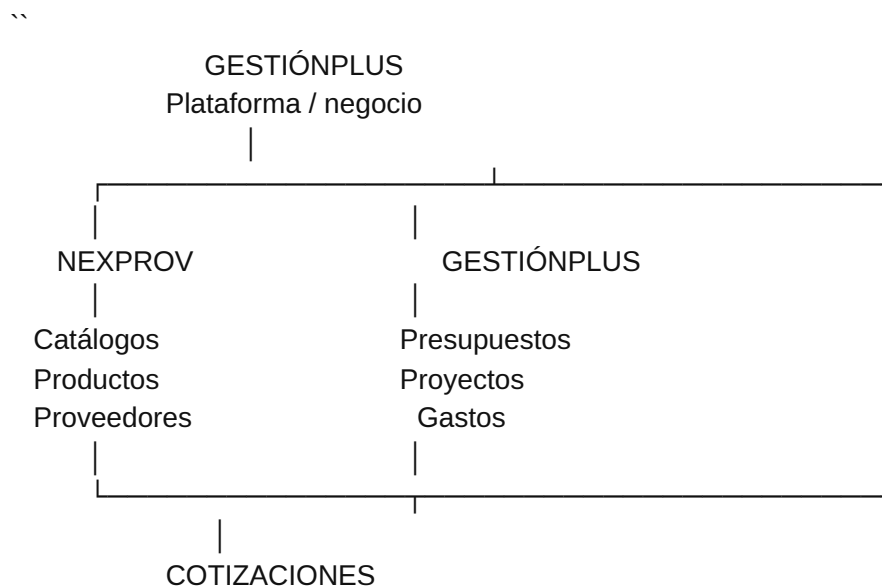
Tú #15

Cotizaciones donde debería ir

ChatGPT #16

Yo pondría Cotizaciones como parte del núcleo compartido, pero con una diferencia importante entre NexProv y GestiónPlus.

Mi propuesta



Pero la cotización no debería "pertener" a una de las dos aplicaciones.

Debería pertenecer al dominio empresarial compartido.

¿Por qué?

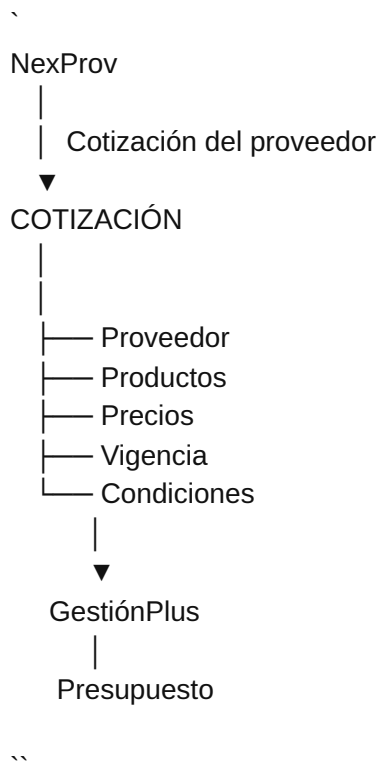
Porque una cotización puede originarse desde NexProv:

Proveedor → productos → precios → cotización

Y posteriormente utilizarse en GestiónPlus:

Cotización → conceptos → presupuesto

Por ejemplo:



Así NexProv descubre/gestiona la oferta, mientras que GestiónPlus utiliza esa información para construir y gestionar el presupuesto.

Entonces yo dividiría las responsabilidades así

Elemento	NexProv	GestiónPlus
-----	-----	-----
Catálogo de productos	Principal	Consume
Proveedores	Principal	Consume
Precios	Principal	Consume
Cotizaciones de proveedores	Principal	Consume/usa
Presupuestos	—	Principal
SPP	—	Principal
Operación del presupuesto	—	Principal

Pero hay una sutileza:

"Cotización" puede significar dos cosas distintas.

1. Cotización recibida de un proveedor
→ Esto encaja naturalmente en NexProv.
2. Cotización que tú elaboras para tu propio cliente

→ Esto encaja en GestiónPlus, y puede terminar convirtiéndose en presupuesto.

Por eso no pondría simplemente un módulo llamado _Cotizaciones_ en cualquiera de las dos apps sin definir primero quién está cotizando a quién.

Y aquí creo que podemos encontrar una pieza clave para terminar de definir qué es GestiónPlus vs NexProv.

Tú #17

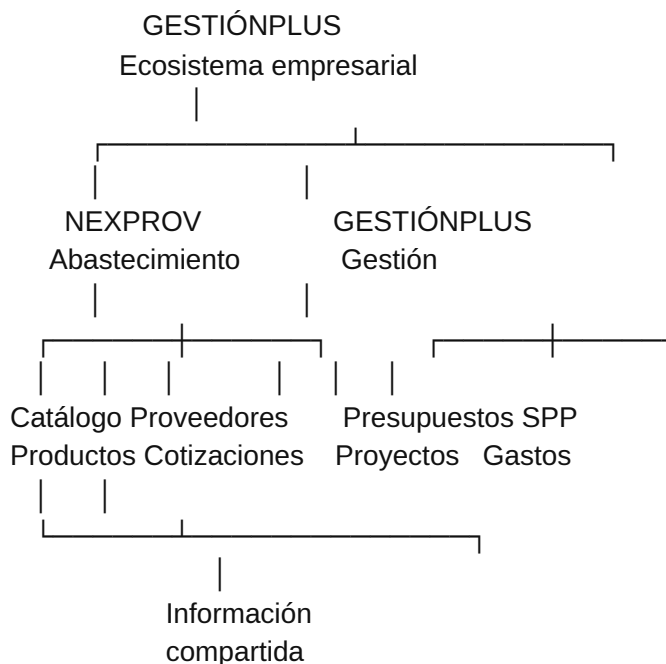
Ok me gusta la idea pero nexprov solo recepción de cotizaciones?

ChatGPT #18

No. NexProv no debería limitarse a recibir cotizaciones. Si lo hacemos así, estaríamos reduciendo demasiado su función.

Yo lo plantearía como el lado de abastecimiento/proveedores del ecosistema, mientras GestiónPlus es el lado de gestión del negocio/presupuesto.

..



,

NexProv podría hacer

1\ Catálogos

,

Productos

- |— SKU
- |— Descripción
- |— Unidad

- Categoría
- Marca

,

2\ Proveedores

,

Proveedor

- Datos fiscales
- Contactos
- Productos
- Condiciones

,

3\ Recepción de cotizaciones

Por ejemplo, un proveedor manda:

,

Cotización #COT-2381

Proveedor: ABC Materiales

Cemento \$215

Varilla \$185

Arena \$80

,

NexProv recibe/importa esa información.

4\ Normalización de información

Esta parte me parece especialmente importante.

Si tres proveedores llaman al mismo producto de formas diferentes:

,

"Cemento gris 50kg"

"Cemento Portland 50 Kg"

"CEMENTO GRIS PORTLAND"

,

NexProv puede relacionarlos con un mismo producto del catálogo.

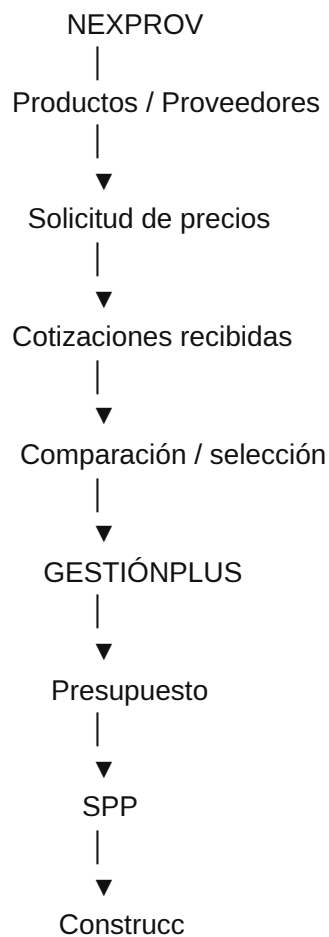
Ahí empieza a tener muchísimo valor.

5\ Comparación de proveedores

	Proveedor A	Proveedor B	Proveedor C
Cemento 50kg	\$215	\$208	\$220
Varilla 3/8	\$185	\$192	\$179
Arena	\$80	\$75	\$83

Y entonces GestiónPlus puede utilizar esa información para el presupuesto.

El flujo que me gusta más



Esto crea una división muy natural:
 NexProv busca y organiza la oferta.
 GestiónPlus transforma esa información en procesos de negocio.

Y Construcc/SPP ejecuta o acompaña la operación.

Eso me parece bastante más coherente que hacer de NexProv simplemente un "buzón de cotizaciones".

Tú #19

El punto 4 explica da explica

ChatGPT #20

Claro. El punto 4, normalización de información, es probablemente una de las piezas más importantes si quieres que NexProv tenga verdadero valor.

¿Qué significa normalizar?

Significa que NexProv entiende que diferentes proveedores pueden estar hablando del mismo producto aunque lo escriban de manera diferente.

Por ejemplo, recibes estas cotizaciones:

..

Proveedor A

"Cemento gris 50 kg"

\$215

Proveedor B

"Cemento Portland Gris 50Kg"

\$208

Proveedor C

"CEMENTO GRIS PORTLAND 50 KG"

\$220

,

Para una persona es evidente que probablemente se trata del mismo producto.

Pero para una computadora:

,

"Cemento gris 50 kg"

≠

"Cemento Portland Gris 50Kg"

≠

"CEMENTO GRIS PORTLAND 50 KG"

,

Son tres cadenas de texto diferentes.

NexProv podría crear un producto maestro

Por ejemplo:

,

PRODUCTO

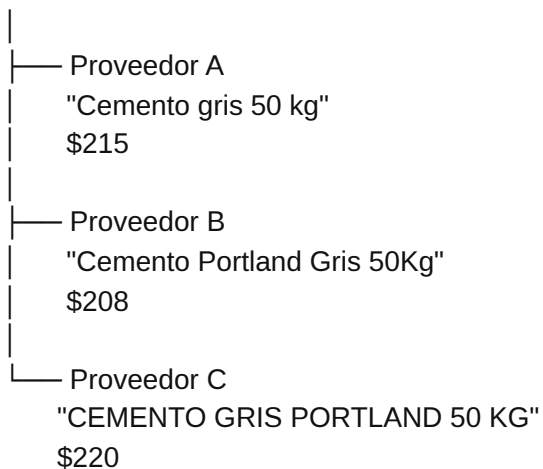
ID: 125
Nombre: Cemento gris
Marca: Portland
Presentación: 50 kg
Unidad: saco
Categoría: Materiales

,

Y después relacionar las diferentes formas en que los proveedores lo manejan:

,

PRODUCTO #125



,

Entonces NexProv sabe:

Son diferentes descripciones comerciales del mismo producto.

¿Para qué sirve?

Aquí es donde se pone interesante.

Supongamos que una empresa necesita:

100 sacos de cemento gris de 50 kg.

GestiónPlus podría consultar a NexProv:

,

Producto:

Cemento gris Portland 50 kg

Cantidad:

100

,

NexProv podría responder:

,

Proveedor	Precio	Total
Proveedor B	\$208	\$20,800
Proveedor A	\$215	\$21,500
Proveedor C	\$220	\$22,000

,

Y entonces GestiónPlus puede utilizar esa información para construir el presupuesto.

Pero hay un segundo nivel

La normalización no necesariamente significa que NexProv automáticamente decida que dos productos son iguales.

Eso sería peligroso.

Por ejemplo:

,

Cemento gris 50 kg

Cemento gris 25 kg

,

No son el mismo producto.

Tampoco:

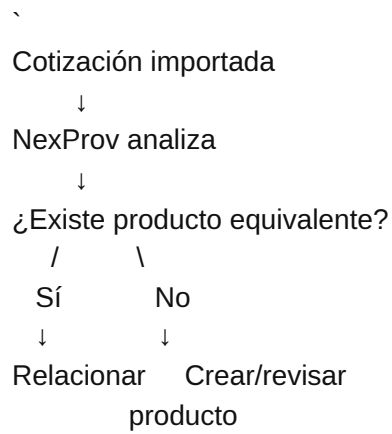
,

Tubo PVC 3"

Tubo PVC 4"

,

Por eso puedes tener un proceso de:



Incluso podrías permitir que un usuario confirme:
"¿Quieres relacionar este artículo con Cemento Gris Portland 50 kg?"

Sí / No

Y aquí aparece algo todavía más potente

Con suficientes cotizaciones, NexProv podría empezar a construir un histórico de precios.

、
Cemento gris 50 kg

Proveedor A
Junio \$205
Julio \$210
Agosto \$215
Septiembre \$215

Proveedor B
Junio \$198
Julio \$202
Agosto \$208
Septiembre \$208

、

Entonces ya no solamente tienes:
"Una cotización."

Tienes inteligencia sobre precios de compra.

Y eso puede alimentar directamente a GestiónPlus.

En otras palabras

NexProv podría convertirse en:

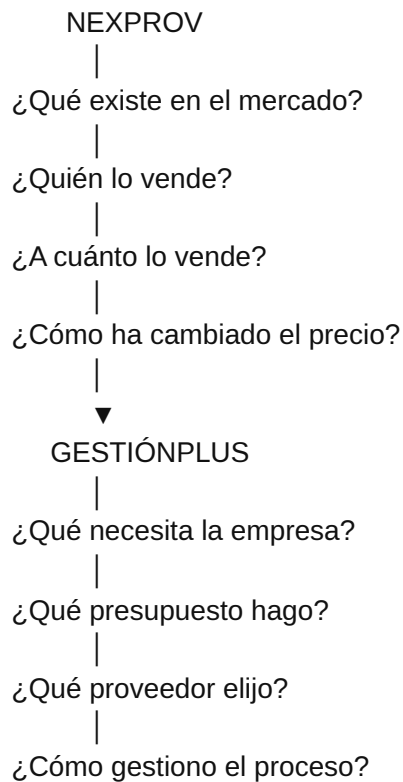
La capa de conocimiento de productos, proveedores y precios del ecosistema.

Mientras que:

GestiónPlus utiliza ese conocimiento para gestionar presupuestos y procesos empresariales.

Eso hace que los dos "gemelos" tengan una razón clara para existir:

,



..

Ahí ya empieza a aparecer una arquitectura de producto bastante más definida que simplemente "GestiónPlus tiene presupuestos y NexProv tiene catálogos".

Tú #21

No entiendo la utilidad

ChatGPT #22

Sí, y de hecho si no ves la utilidad, probablemente no deberíamos meter esa función todavía. Te lo explico con un ejemplo mucho más aterrizado.

Supongamos que tienes una empresa constructora

Tu empresa necesita:
100 sacos de cemento.

En NexProv tienes tres proveedores:

Proveedor	Cómo lo llama	Precio
-----	-----	-----
Cemex	Cemento gris 50 kg	\$215
Proveedor B	Cemento Portland 50Kg	\$208
Proveedor C	Cemento gris Portland	\$220

La "normalización" solamente serviría para que NexProv pueda entender:
Estos tres artículos probablemente representan el mismo producto.

Entonces podría mostrarte:

Cemento gris Portland 50 kg

- * Proveedor A → \$215
- * Proveedor B → \$208
- * Proveedor C → \$220

Y tú podrías elegir el de \$208.

Pero aquí está el punto

Si NexProv ya recibe cotizaciones y tú manualmente sabes qué productos corresponden,
entonces la normalización no te aporta prácticamente nada hoy.

No necesitas construir un sistema complejo para resolver un problema que todavía no tienes.

Yo dejaría NexProv mucho más sencillo

Con lo que me has contado, inicialmente lo veo así:

^^

NEXPROV

|

|— Proveedores

|

|— Catálogo de productos

|

|— Importar productos

|

|— Recibir/importar cotizaciones

|

└─ Consultar cotizaciones

,

Y GestiónPlus:

,

GESTIÓNPLUS

|
├─ Presupuestos
├─ Cotizaciones
└─ SPP

,

Pero hay que definir qué significa "cotización" en cada aplicación.

Porque ahí sí hay una distinción útil:

NexProv

Cotización que recibes de un proveedor.

"Proveedor ABC me cotiza 100 sacos de cemento a \$208."

GestiónPlus

Cotización/presupuesto que tú preparas para tu cliente.

"Le voy a cotizar al cliente 100 sacos de cemento a \$250."

Entonces:

,

PROVEEDOR

|
└─ cotiza
▼

NEXPROV

|
└─ precio de compra
▼

GESTIÓNPLUS

|
└─ arma presupuesto/cotización
▼

CLIENTE

..

Ese flujo sí me parece útil y concreto.

Y además explica mucho mejor por qué existen dos aplicaciones gemelas compartiendo la misma API.

Tú #23

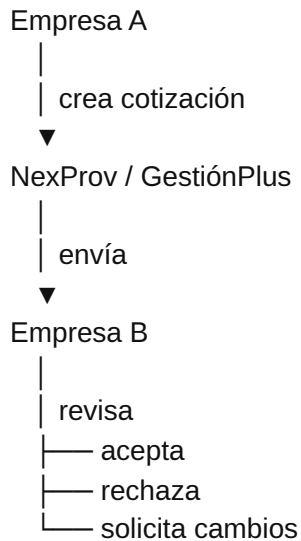
Las cotizaciones no son de varios proveedores son hacia una empresa

ChatGPT #24

Ah, entonces sí cambia el concepto. Yo estaba interpretando "cotización" como cotización de proveedor, y en tu sistema no es eso.

Si las cotizaciones son de una empresa hacia otra empresa, entonces el flujo correcto es:

``



,

Por ejemplo:

Constructora ABC necesita 500 m² de piso.

Una empresa que vende ese material genera:

,

Cotización #COT-001

Cliente: Constructora ABC

Piso cerámico	500 m ²	\$180
---------------	--------------------	-------

Adhesivo	50 bultos	\$250
----------	-----------	-------

Subtotal

IVA

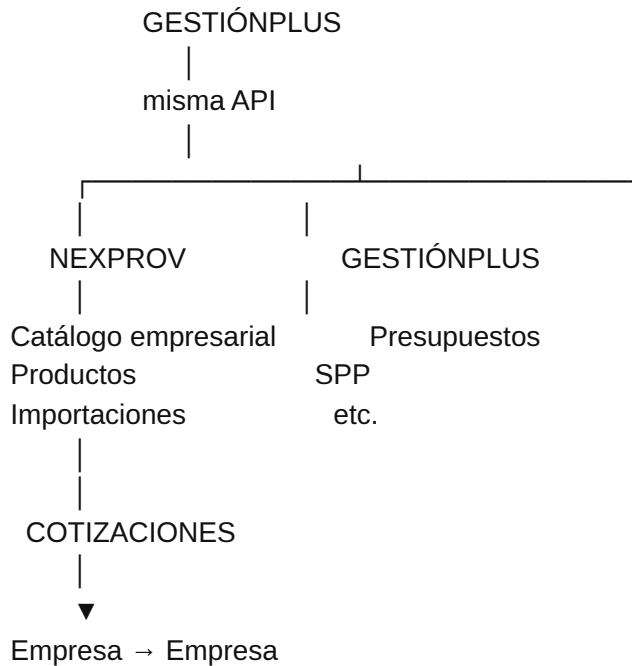
Total

Eso sí es una cotización comercial.

Entonces retiro la idea anterior

No veo sentido en que NexProv tenga como función principal "recibir cotizaciones de proveedores".

Más bien, por lo que ahora me explicas, la relación puede ser:



Y aquí aparece una posibilidad mucho más clara:

NexProv puede ser el lado comercial

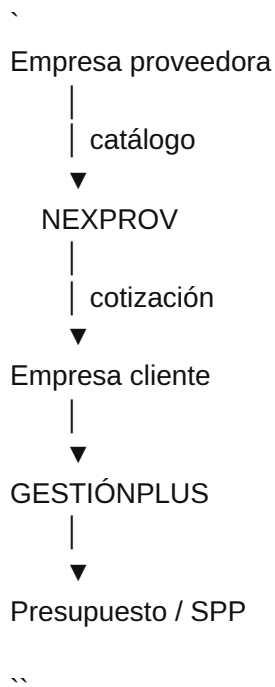
Una empresa publica/maneja:

- * sus productos
- * sus servicios
- * sus precios
- * sus catálogos
- * sus cotizaciones

Y otra empresa puede recibir esa información.

Mientras que GestiónPlus puede encargarse de utilizar esa información dentro de sus procesos de presupuesto.

Por ejemplo:



Eso me parece mucho más coherente con lo que realmente tienes.

Y ahora sí hay una pregunta de diseño que vale la pena resolver: ¿quién crea actualmente las cotizaciones en NexProv: la empresa que vende, la empresa que compra, o ambas? Esa respuesta nos va a definir exactamente el papel de NexProv frente a GestiónPlus.

Tú #25

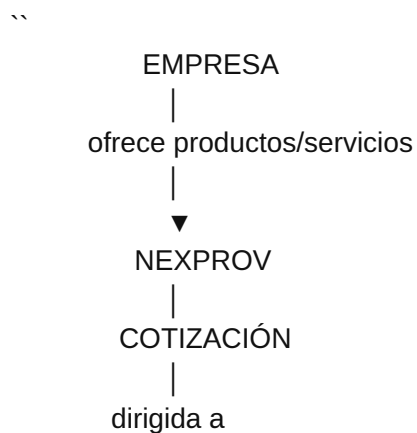
Cliente empresa. Míralo mejor así

ChatGPT #26

Sí. Con esa corrección, hay que cambiar completamente el modelo mental.

"Cliente empresa" significa que la cotización está dirigida a una empresa cliente. No estamos hablando de cotizaciones internas de proveedores.

Entonces lo veo así:



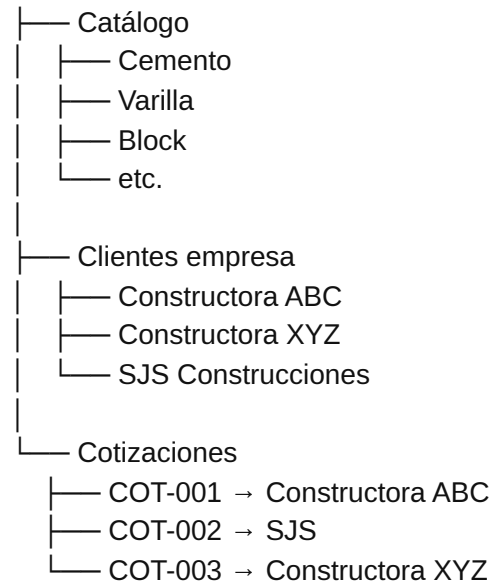
▼
CLIENTE EMPRESA

Ejemplo

Una empresa llamada Materiales del Norte utiliza NexProv.

Tiene:

NexProv



La empresa selecciona productos de su catálogo:

Cemento 100 × \$210
Varilla 200 × \$185
Block 1000 × \$14

y genera:

COT-001
Cliente: Constructora ABC

Subtotal: \$...

IVA: \$...

Total: \$...

La manda a Constructora ABC.

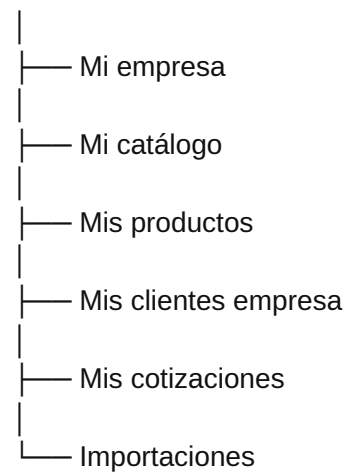
Entonces NexProv tiene mucho más sentido

Yo ahora lo veo como:

NexProv = plataforma comercial entre empresas.

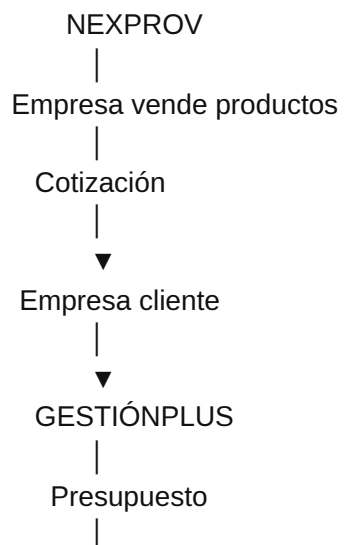
Su centro sería:

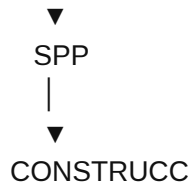
NEXPROV



Mientras que GestiónPlus puede ser el lado de la empresa que gestiona sus necesidades, presupuestos y procesos.

Por ejemplo:





..

Pero hay algo todavía más importante:

No asumiría que NexProv es "para proveedores".

El nombre puede llevar a esa interpretación, pero funcionalmente parece más bien: un sistema de gestión comercial B2B.

Es decir, una empresa usa NexProv para presentar su oferta y cotizar a otras empresas.

Eso hace que la relación entre ambas aplicaciones empiece a quedar bastante más clara.

Tú #27

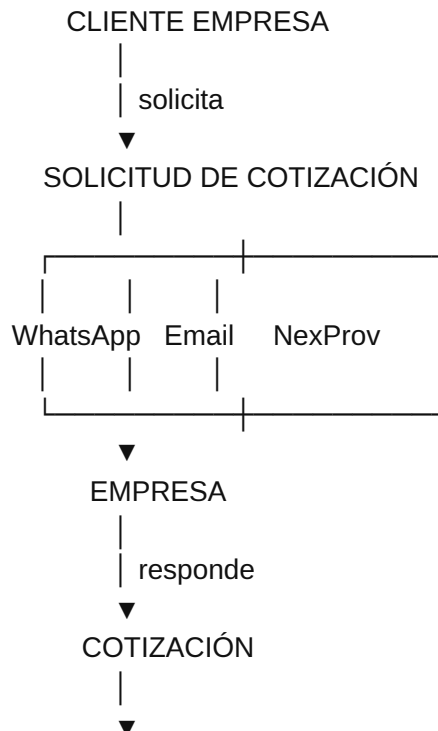
Cliente solicita y empresa contesta por diferentes vías

ChatGPT #28

Ahora sí. Con eso ya se entiende mucho mejor el concepto.

La cotización nace por una solicitud del cliente empresa y la empresa proveedora responde esa solicitud, independientemente del medio por el que llegue.

..



CLIENTE EMPRESA

Y aquí veo una distinción fundamental:

Solicitud $\neq$ Cotización

La solicitud viene del cliente:

"Necesito 100 sacos de cemento, 50 varillas y 20 bultos de adhesivo. ¿Cuánto me cuesta?"

La cotización es la respuesta de la empresa:

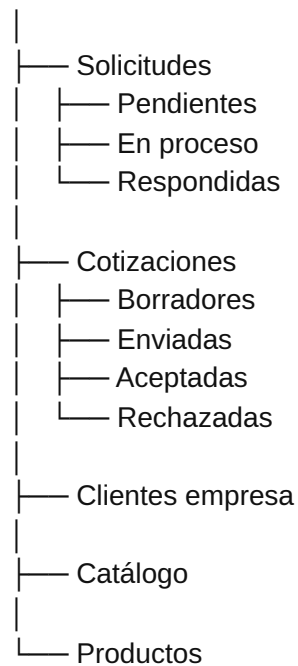
"Te puedo proporcionar eso por \$XX,XXX, con estas condiciones."

Eso cambia bastante cómo yo diseñaría NexProv.

NexProv

Sería el lugar donde la empresa puede recibir, organizar y responder solicitudes comerciales.

NEXPROV



Y las solicitudes pueden entrar por:

WhatsApp

Email

Teléfono

NexProv

Formulario web

Importación

,

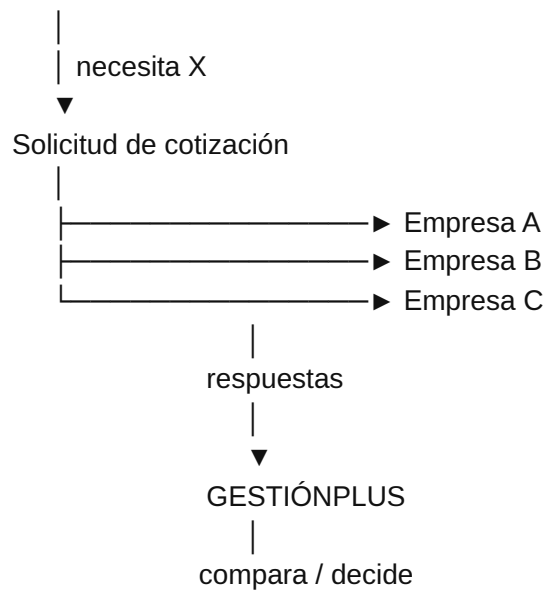
NexProv no necesariamente necesita controlar todos esos canales. Puede simplemente registrar que la solicitud llegó por determinado medio.

Y aquí aparece algo muy interesante para GestiónPlus

Si una empresa cliente utiliza GestiónPlus y necesita comprar algo:

,

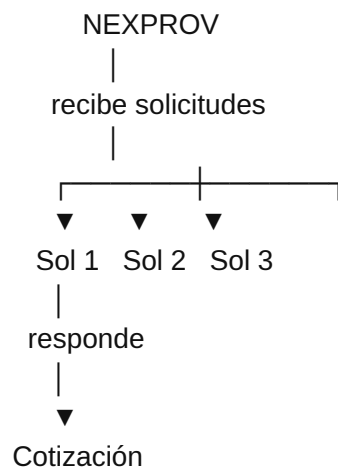
GESTIÓNPLUS



,

Mientras que NexProv representa el otro lado:

,



..

Entonces los "gemelos" empiezan a tener una lógica muy buena:

GestiónPlus

"Necesito algo."

NexProv

"Puedo ofrecerte esto."

Y ambos pueden compartir la misma API porque están participando en el mismo proceso comercial B2B, pero desde lados diferentes.

Ese concepto me parece bastante más sólido que pensar en NexProv simplemente como "catálogo + cotizaciones".

Tú #29

NEXPROV | ├── Solicitudes | ├── Pendientes | ├── En proceso | └── Respondidas |
| ├── Cotizaciones | ├── Borradores | ├── Enviadas | ├── Aceptadas | └── Rechazadas
| ├── Clientes empresa | ├── Catálogo | └── Productos Este me gusto además me
gustaría que en gestionapp estuviera el concentrado de archivos para SPP pptos y
cotizaciones

ChatGPT #30

Sí. Ahí ya empieza a aparecer una arquitectura de producto coherente. Y la idea del concentrado de archivos en GestiónPlus me parece especialmente buena porque evita repartir documentos entre NexProv, SPP y Presupuestos.

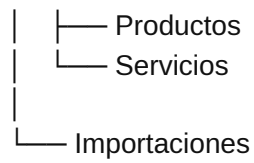
Yo lo estructuraría así:

NexProv — lado comercial

``

NEXPROV

```
|
| ├── Solicitudes
| | ├── Pendientes
| | ├── En proceso
| | └── Respondidas
| ├── Cotizaciones
| | ├── Borradores
| | ├── Enviadas
| | ├── Aceptadas
| | └── Rechazadas
| ├── Clientes empresa
| └── Catálogo
```



,

Su pregunta principal sería:

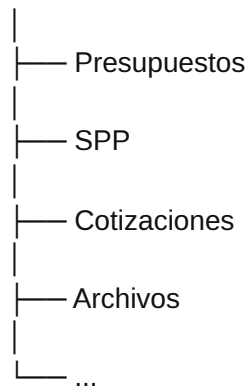
¿Qué me están solicitando y qué le puedo ofrecer a ese cliente?

GestiónPlus — lado de gestión

Aquí yo agregaría:

,

GESTIÓNPLUS



,

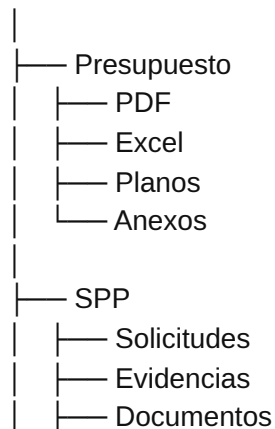
Pero Archivos no lo haría como un simple explorador de carpetas.

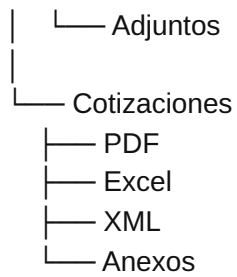
Lo convertiría en un concentrador documental relacionado con los procesos.

Por ejemplo:

,

ARCHIVOS



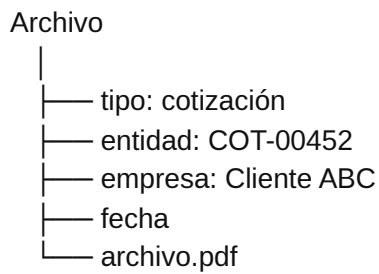


,

Pero lo importante es que el archivo mantenga su relación con el objeto de negocio.

Por ejemplo:

,



,

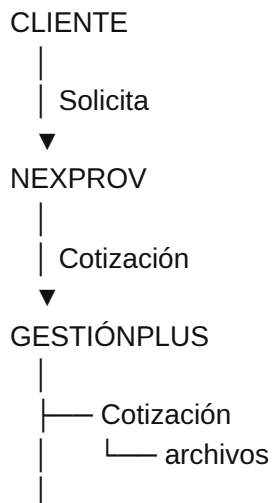
Entonces desde una cotización puedes ver:
Documentos relacionados

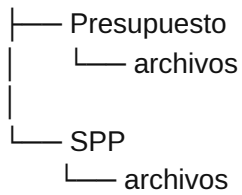
y desde Archivos puedes encontrarla.

Lo haría todavía más integrado

Imagina este proceso:

,





,

Y todo queda relacionado.

Por ejemplo:

Cotización COT-00234

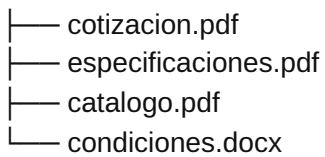
,

Cotización COT-00234

Cliente: Constructora ABC

Total: \$125,000

Documentos (4)

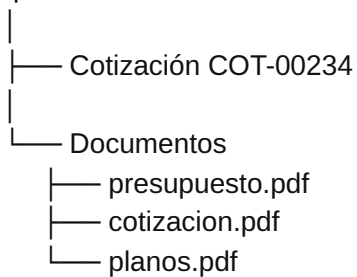


,

Después esa cotización genera o alimenta un presupuesto:

,

Presupuesto PRES-00127

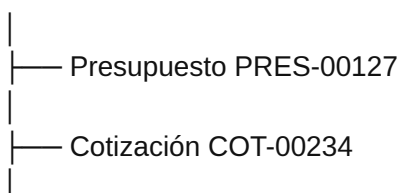


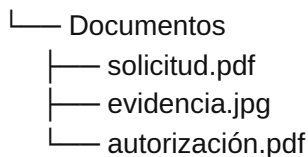
,

Y posteriormente puede relacionarse con un SPP:

,

SPP-0081





..

Eso genera una idea bastante potente

En lugar de que GestiónPlus sea:

"la app donde hago presupuestos"

se convierte en:

el lugar donde una empresa concentra y da seguimiento a sus procesos y documentos comerciales/operativos.

Y NexProv sería el punto de interacción comercial entre empresas.

La división empieza a quedar así

NexProv	GestiónPlus	
-----	-----	-----
Solicitudes	Principal	Puede recibir relacionadas
Clientes empresa	Principal	Consume
Catálogo	Principal	Consume
Productos	Principal	Consume
Cotizaciones	Principalmente comercial	Gestiona/consulta
Presupuestos	—	Principal
SPP	—	Principal
Archivos	—	Concentrador documental
Construcc	—	Interfaz operativa del SPP

Y algo que cuidaría desde ahora: no duplicaría los archivos físicamente entre aplicaciones. La API debería manejar un sistema documental común y cada aplicación simplemente mostrar los archivos que le corresponden.

Eso además encaja muy bien con que NexProv y GestiónPlus sean gemelos: dos experiencias diferentes sobre el mismo núcleo de datos y procesos.

Tú #31

Bien delegaciones de casa app

ChatGPT #32

Sí. Si por "delegaciones de cada app" te refieres a qué responsabilidad debe tener cada aplicación, yo las dejaría muy delimitadas para evitar que terminen haciendo lo mismo.

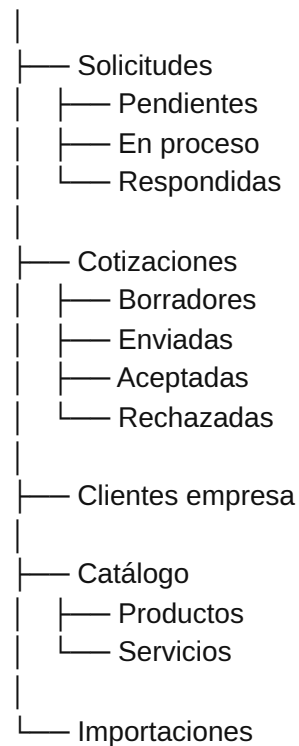
1\ NexProv — relación comercial

NexProv responde a:

“¿Qué me están solicitando y qué voy a ofrecer?”

..

NEXPROV



,

Responsabilidades de NexProv

- * Gestionar clientes empresa.
- * Mantener el catálogo comercial.
- * Recibir/importar información de productos.
- * Recibir solicitudes de clientes.
- * Preparar y enviar cotizaciones.
- * Dar seguimiento al estado de las cotizaciones.
- * Gestionar la relación comercial empresa ↔ empresa.

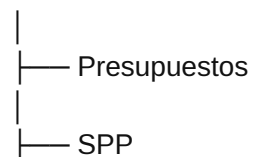
2\ GestiónPlus — gestión empresarial

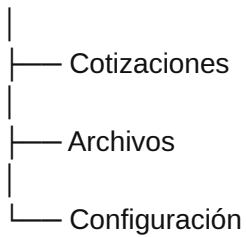
GestiónPlus responde a:

“¿Qué estoy presupuestando, gestionando y documentando?”

,

GESTIÓNPLUS





`

Responsabilidades de GestiónPlus

- * Crear y administrar presupuestos.
- * Gestionar SPP.
- * Consultar cotizaciones relacionadas.
- * Relacionar:

`

Cotización
↓
Presupuesto
↓
SPP

`

- * Concentrar los archivos relacionados con esos procesos.
- * Mantener el historial documental.
- * Servir como sistema de gestión interna de la empresa.

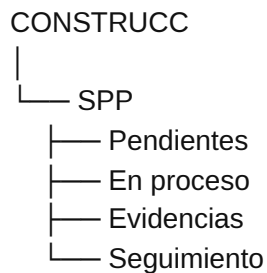
3\ Construcc — ejecución

Aquí yo mantendría a Construcc todavía más especializada.

Construcc responde a:

“¿Qué tengo que ejecutar y qué está pasando en campo/operación?”

`



`

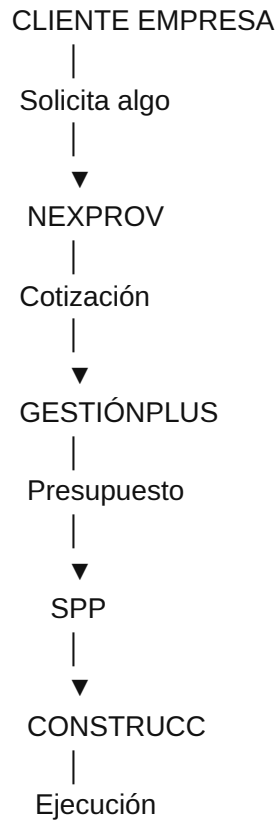
No le metería catálogo, clientes, cotizaciones, etc., salvo que posteriormente exista una razón

clara.

4\ La relación entre las tres

Aquí está la parte que me parece más importante:

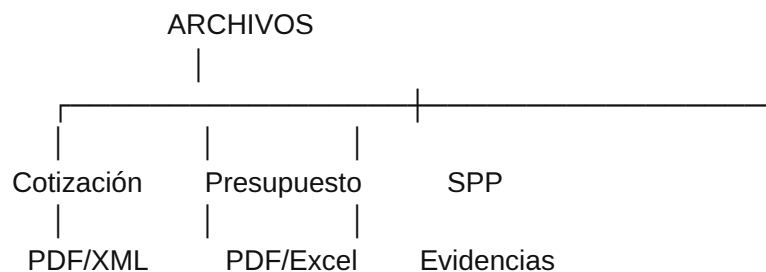
,



,

Y los documentos acompañan todo el proceso:

,



..

La regla que usaría

NexProv vende/negocia.

GestiónPlus administra/gestiona.

Construcc ejecuta.

Y la API de GestiónPlus es el núcleo compartido.

Eso evita un problema muy común: que dentro de un año tengas _Cotizaciones en NexProv_, _Cotizaciones en GestiónPlus_, _Cotizaciones en Construcc_ y nadie sepa cuál es la fuente de verdad. La entidad existe una sola vez; cada aplicación tiene su propia vista y responsabilidad sobre ella.

Tú #33

Nexprov no debe tener clientes

ChatGPT #34

Correcto. Entonces NexProv no debe manejar “Clientes” como módulo propio. Eso estaba mezclando responsabilidades.

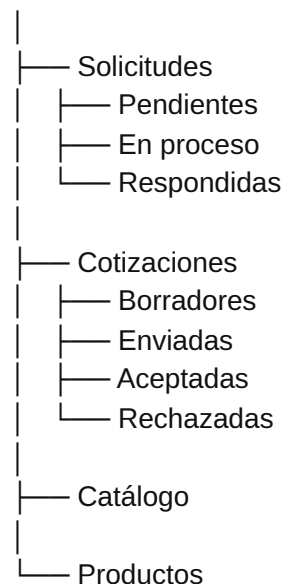
La separación quedaría mejor así:

NEXPROV

Su función es recibir y gestionar solicitudes comerciales y generar respuestas/cotizaciones.

..

NEXPROV



,

No tendría:

*  Clientes

*  Usuarios de clientes

- * ~~CRM~~
- * ~~Gestión de relaciones con clientes~~

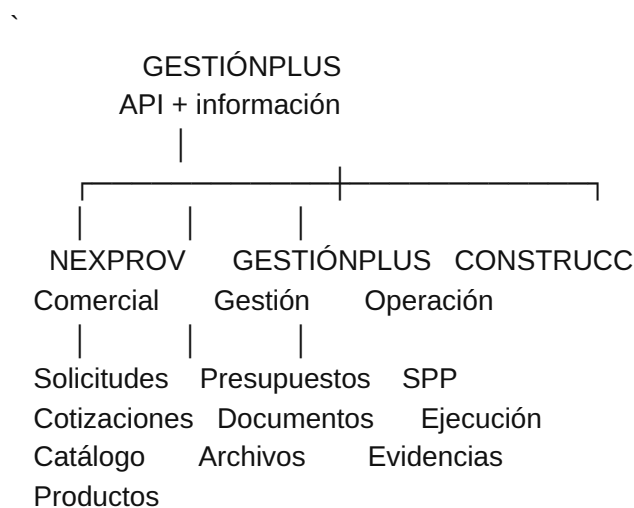
La empresa que usa NexProv es quien responde solicitudes y genera cotizaciones. El hecho de que una solicitud venga de otra empresa no significa que NexProv tenga que convertirse en un CRM.

Entonces, ¿qué representa NexProv?

Más concretamente:

NexProv es el punto comercial donde una empresa recibe solicitudes y prepara respuestas/cotizaciones utilizando su catálogo y productos.

Y GestiónPlus queda como el núcleo de gestión:



..

Además, esto refuerza algo importante: NexProv y GestiónPlus pueden compartir datos sin que ambos tengan que administrar las mismas cosas.

Si una solicitud/cotización necesita identificar a la contraparte, esa relación puede existir en el modelo de negocio/API, sin convertir “Clientes” en un módulo de NexProv.

Tú #35

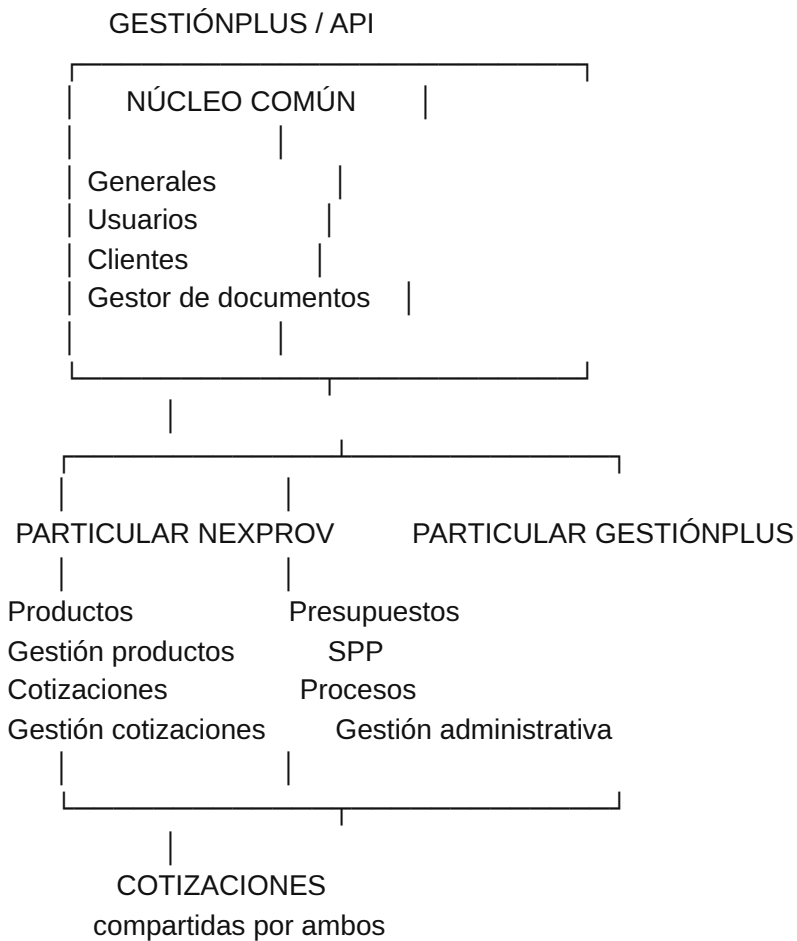
El núcleo de la empresa es común. Generales Usuarios Clientes Gestor de documentos
Particular Nexprov: productos y su gestión. Cotizaciones y su gestión completa Gestiónplus: el resto y también Cotizaciones Ahí me surge la cuestión de configuración para los delegaciones de nexprov desde gestión

ChatGPT #36

Sí. Con esa definición cambia bastante la arquitectura, y la cuestión de delegaciones de NexProv tiene sentido que viva en GestiónPlus.

La separación que planteas la veo así:

..



,

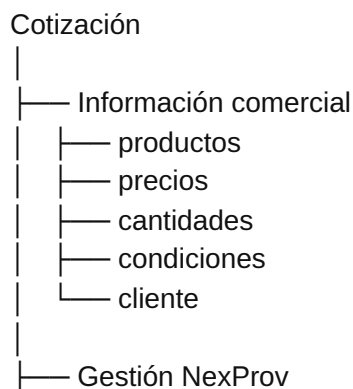
Y aquí está el punto importante

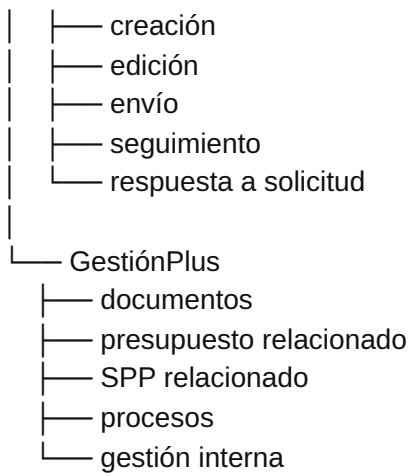
Si Cotizaciones pertenece funcionalmente a ambos, no conviene que cada aplicación tenga su propia implementación.

Debe existir una sola Cotización en el núcleo/API, y cada aplicación aporta una parte de su gestión.

Por ejemplo:

,





,

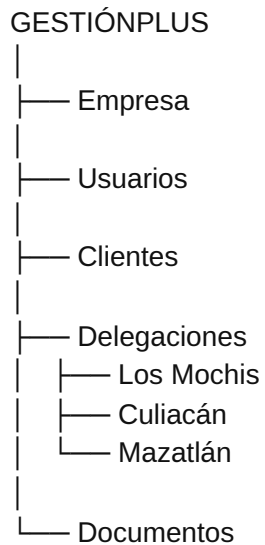
Ahora sí: Delegaciones de NexProv

Yo sí pondría la configuración de delegaciones en GestiónPlus.

Porque una delegación no es realmente una característica exclusiva de NexProv. Es una estructura organizacional de la empresa.

Por ejemplo:

,



,

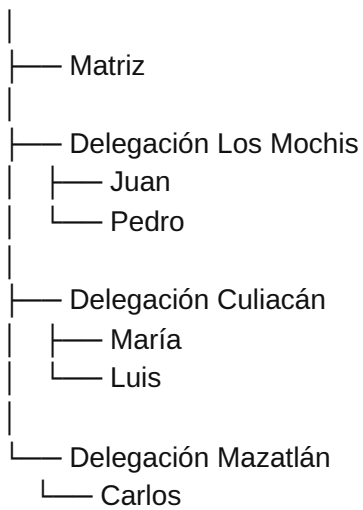
Y después NexProv simplemente utiliza esa estructura.

Ejemplo

Una empresa tiene:

,

EMPRESA



,

Desde GestiónPlus → Configuración → Delegaciones se define:

- * Nombre
- * Dirección
- * Datos de contacto
- * Responsable
- * Usuarios pertenecientes
- * Clientes asignados, si aplica
- * Documentos relacionados
- * Estado activa/inactiva

Y NexProv consume esa información.

Entonces en NexProv puedes tener:

,

NEXPROV

Solicitudes

Cotizaciones

Productos

Gestión de productos

Delegación actual:

[Los Mochis ▼]

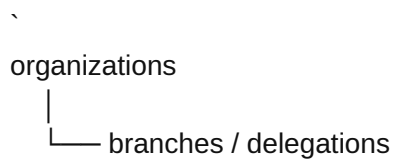
,

Y al cambiar de delegación, NexProv filtra la información correspondiente.

Pero hay una decisión arquitectónica importante

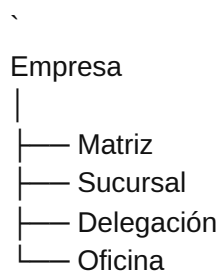
Yo no llamaría necesariamente a esto "Delegaciones de NexProv" en la base de datos.

Lo modelaría como algo más general:



Porque mañana GestiónPlus podría necesitar exactamente la misma estructura.

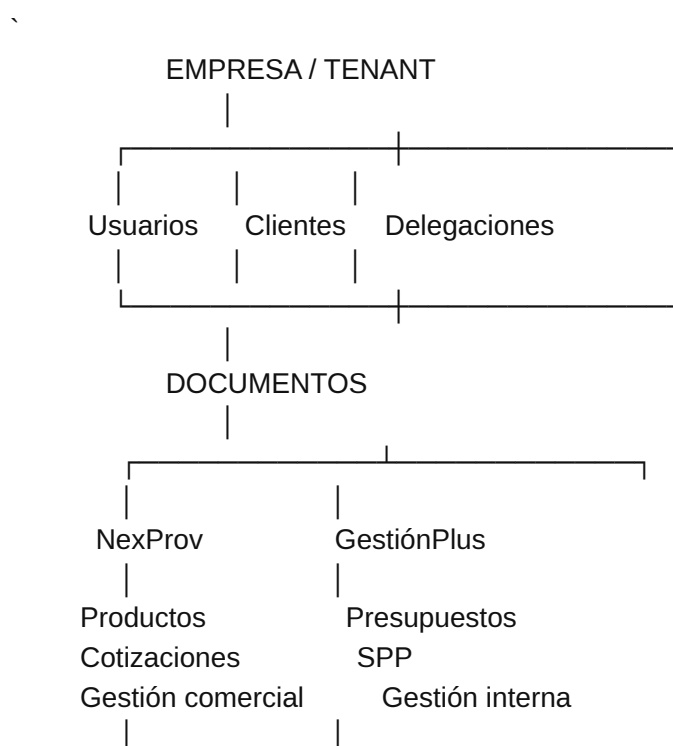
Por ejemplo:



Y entonces NexProv no "crea" delegaciones. Las consume.

Eso encaja muy bien con tu idea de que existe un núcleo común.

En términos de arquitectura





Y esto además conecta directamente con multi-tenancy: primero defines claramente Empresa → Delegaciones → Usuarios → Clientes → documentos y operaciones, y después determinas qué entidades llevan `tenant_id`, `delegation_id`, etc.

La siguiente cuestión que yo resolvería es qué significa exactamente una delegación para NexProv: si solamente sirve para separar usuarios/productos/cotizaciones, o si también debe tener su propio catálogo, precios, clientes y numeración de cotizaciones. Esa decisión cambia bastante el modelo de datos.

Tú #37

Lo de configuración no. Pero el resto si elabora un resumen completo

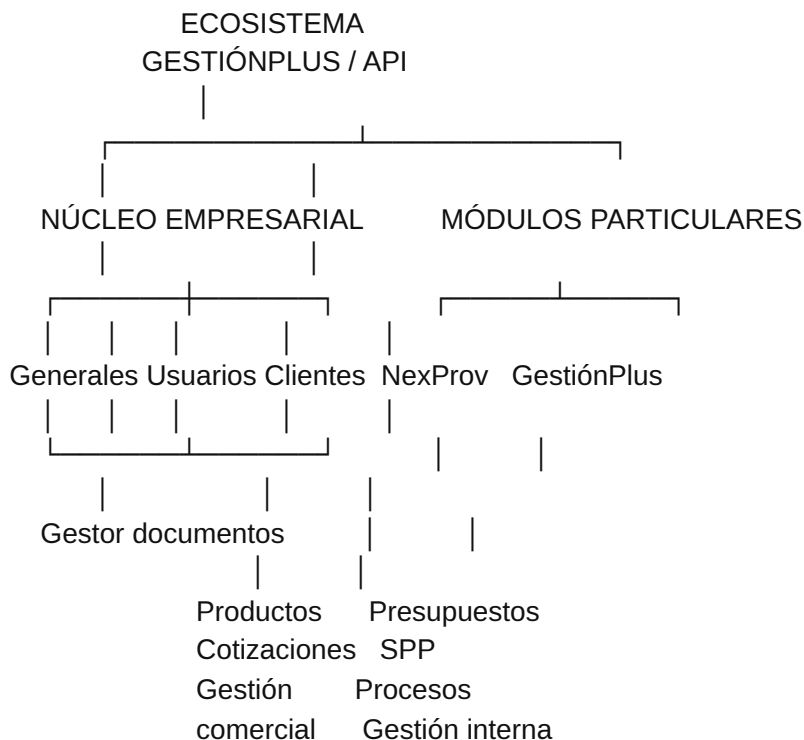
ChatGPT #38

Sí. Quitando por completo la parte de Configuración, la estructura conceptual queda mucho más limpia. Este sería el resumen consolidado de lo que hemos definido hasta ahora.

Ecosistema GestiónPlus + NexProv + Construcc

La idea no es tener tres sistemas independientes, sino un ecosistema con un núcleo empresarial común y aplicaciones especializadas.

..



1\ Núcleo común

El núcleo pertenece a la empresa, no a NexProv ni exclusivamente a GestiónPlus.

Generales

Información general de la empresa y datos necesarios para operar dentro de la plataforma.

Usuarios

Administración de las personas que utilizan el ecosistema.

Los usuarios pueden tener diferentes permisos y acceso a las distintas aplicaciones o funciones.

Clientes

La empresa puede tener registrados sus clientes.

Aquí es importante mantener la distinción:
Cliente es una entidad del núcleo empresarial.

Por eso no pertenece exclusivamente a NexProv.

NexProv puede utilizar clientes, pero no es el dueño conceptual de ellos.

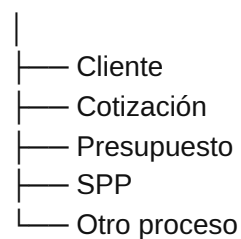
Gestor de documentos

Es otro componente común.

Los documentos deben existir en un repositorio centralizado, pudiendo estar relacionados con diferentes elementos del sistema.

Por ejemplo:

Documento



La ventaja es que el documento no tiene que duplicarse porque sea utilizado desde NexProv y

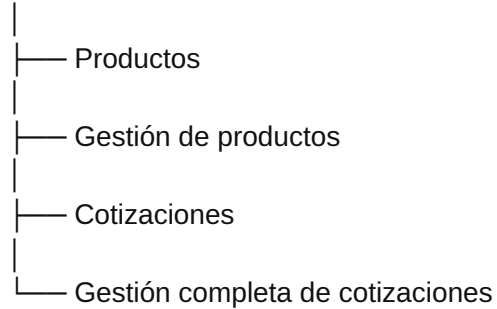
GestiónPlus.

2\ NexProv

NexProv es la aplicación especializada en la parte comercial y de productos.

,

NEXPROV



,

Productos

Aquí vive el catálogo/productos que la empresa maneja comercialmente.

Incluye la información necesaria para trabajar con ellos dentro de NexProv.

Gestión de productos

No es solamente visualizar productos.

Aquí entran las operaciones necesarias para mantenerlos y utilizarlos comercialmente.

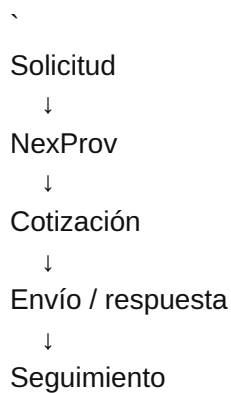
Por ejemplo:

- * altas
- * modificaciones
- * información comercial
- * precios
- * disponibilidad, si posteriormente aplica
- * importaciones
- * relaciones necesarias para cotizar

Cotizaciones

NexProv tiene una participación importante en las cotizaciones porque es donde ocurre gran parte de la actividad comercial.

Por ejemplo:



La cotización puede originarse en respuesta a una solicitud de una empresa.

Y aquí mantenemos la definición que ya establecimos:

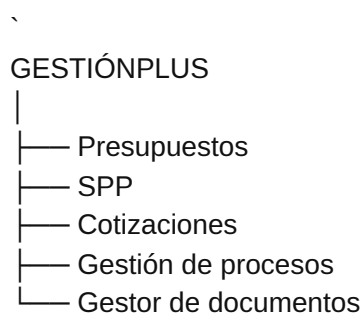
Solicitud \= alguien solicita algo.

Cotización \= la empresa responde ofreciendo productos/servicios bajo determinadas condiciones.

No estamos hablando de comparar cotizaciones de diferentes proveedores.

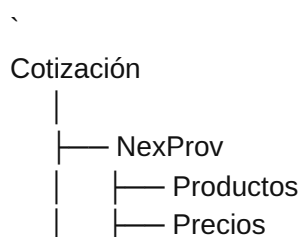
3\ GestiónPlus

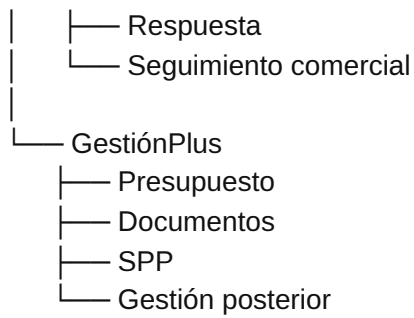
GestiónPlus contiene el resto de la gestión empresarial, además de participar también en las cotizaciones.



Su función es más amplia que la parte estrictamente comercial.

Por ejemplo:





,

Esto no significa que existan dos cotizaciones diferentes.

Debe existir una sola entidad Cotización en el backend, y ambas aplicaciones trabajan sobre ella desde perspectivas diferentes.

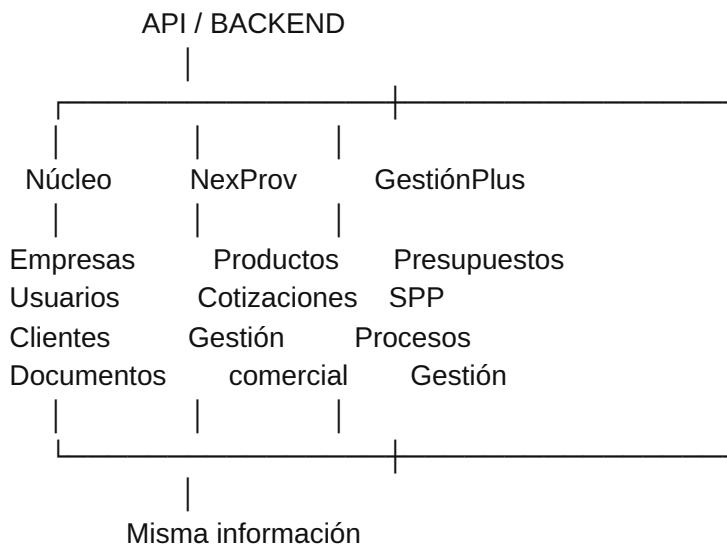
4\ Relación entre NexProv y GestiónPlus

Aquí está probablemente la parte más importante de la arquitectura.

No son dos productos aislados.

Son dos aplicaciones especializadas que utilizan el mismo núcleo empresarial y la misma API.

,



,

Por eso:

- * Un usuario no debería existir dos veces.
- * Un cliente no debería existir dos veces.
- * Un documento no debería existir dos veces.
- * Una cotización no debería existir dos veces.

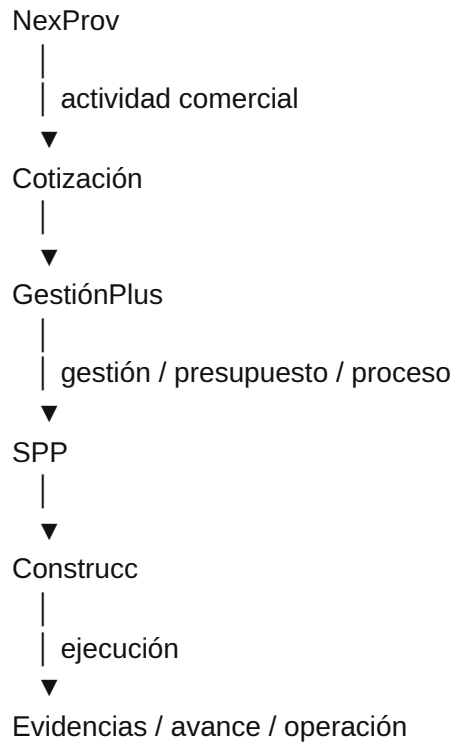
- * Los productos de NexProv pueden utilizarse dentro de las cotizaciones.
- * GestiónPlus puede trabajar posteriormente con esas mismas cotizaciones.

5\ Construcc

Construcc queda como la aplicación orientada a la ejecución/operación.

La relación sería:

,



,

Por lo tanto:

NexProv

¿Qué voy a ofrecer?

Productos, solicitudes y cotizaciones.

GestiónPlus

¿Cómo voy a gestionar lo que estoy haciendo?

Presupuestos, SPP, documentos y procesos.

Construcc

¿Cómo se está ejecutando?

Operación, seguimiento y evidencias.

6\ La empresa es el centro

El concepto fundamental debería ser la empresa.

No NexProv.

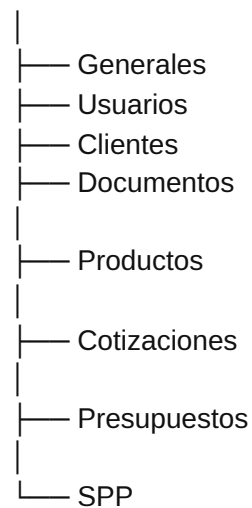
No GestiónPlus.

No Construcc.

La empresa es la que tiene:

,

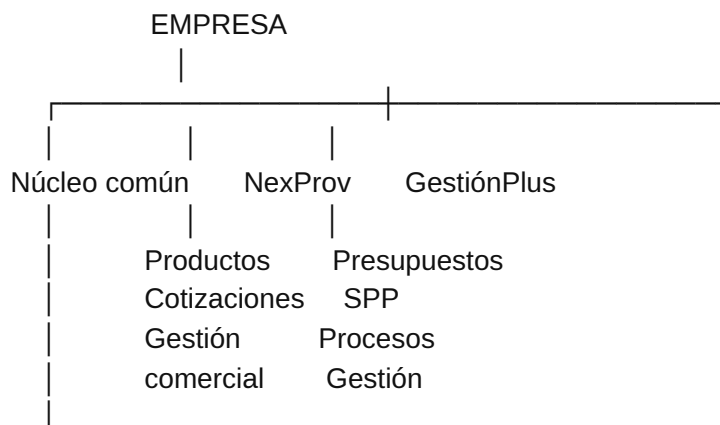
EMPRESA

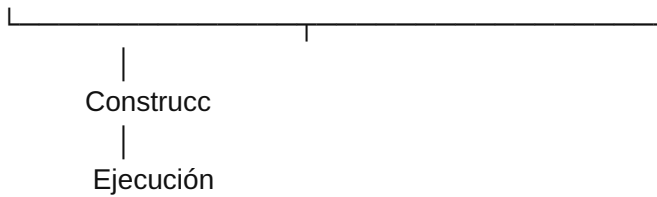


,

Las aplicaciones solamente presentan y gestionan diferentes partes de esa información.

,





7\ Principio que queda definido

La arquitectura conceptual podría resumirse en una frase:

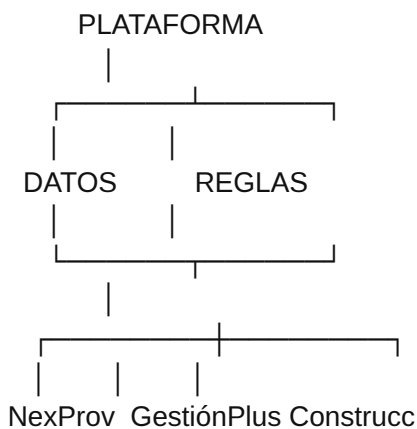
GestiónPlus es el núcleo empresarial y de gestión; NexProv es la especialización comercial y de productos; Construcc es la especialización operativa. Los tres comparten la información empresarial a través de la misma plataforma/API.

Y algo especialmente importante:

No se debe diseñar cada aplicación como si fuera un sistema independiente.

La fuente de verdad debe estar en el backend común.

Eso permite que en el futuro puedas agregar otra aplicación sin tener que duplicar usuarios, clientes, documentos, cotizaciones, etc.



Con esta definición, la siguiente discusión importante ya no es qué módulos agregar, sino definir qué entidades pertenecen al núcleo, quién es dueño de cada entidad y qué relaciones tienen entre sí. Eso es lo que posteriormente te permitirá implementar correctamente la multi-tenancy y evitar que el proyecto se vuelva una colección de módulos desconectados.

Tú #39

En un md

ChatGPT #40

Claro. Te lo dejo como un documento Markdown, listo para guardar como arquitectura-ecosistema.md.

Arquitectura del ecosistema GestiónPlus + NexProv + Construcc

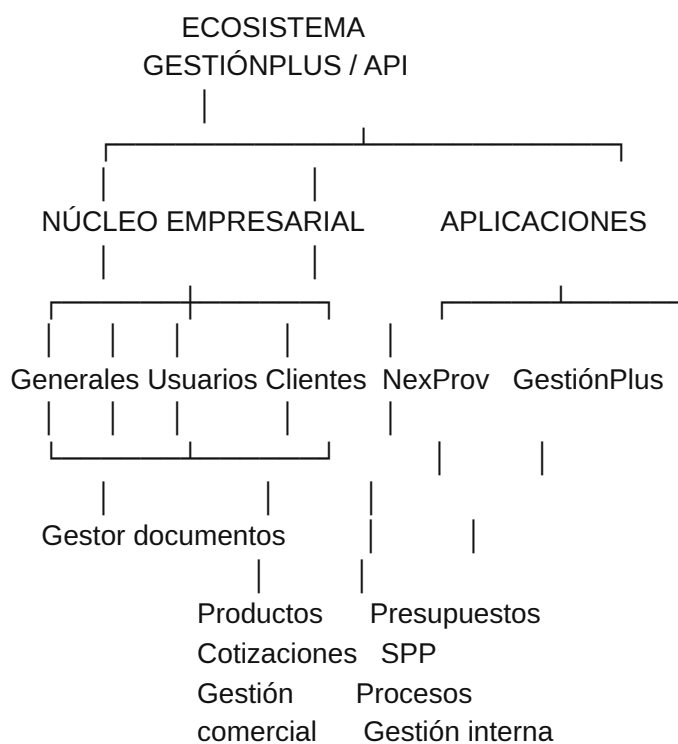
Arquitectura del ecosistema GestiónPlus + NexProv + Construcc

1\ Concepto general

GestiónPlus, NexProv y Construcc forman parte de un mismo ecosistema empresarial.

No deben considerarse como tres sistemas independientes, sino como aplicaciones especializadas que comparten un núcleo empresarial, una API y una misma fuente de información.

--



,

2\ Núcleo empresarial común

El núcleo pertenece a la empresa, no exclusivamente a NexProv ni a GestiónPlus.

Contiene las entidades y funcionalidades comunes que necesitan las diferentes aplicaciones.

2.1 Generales

Información general de la empresa y datos necesarios para operar dentro de la plataforma.

2.2 Usuarios

Administración de las personas que utilizan el ecosistema.

Los usuarios pueden tener diferentes permisos y acceso a diferentes aplicaciones y funcionalidades.

2.3 Clientes

Los clientes forman parte del núcleo empresarial.

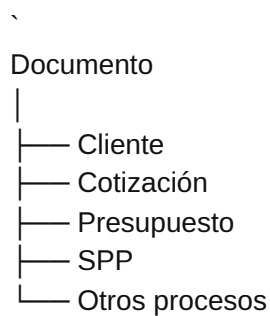
Un cliente no pertenece exclusivamente a NexProv.

Las diferentes aplicaciones pueden utilizar la información de clientes según sus necesidades.

2.4 Gestor de documentos

Repositorio centralizado para los documentos de la empresa.

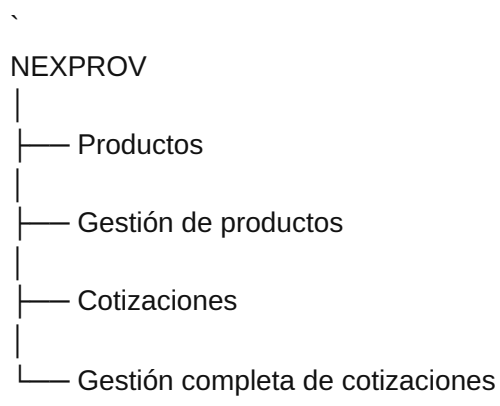
Los documentos pueden estar relacionados con diferentes entidades:



Un mismo documento debe existir una sola vez en el sistema y poder ser relacionado desde las diferentes aplicaciones cuando corresponda.

3\ NexProv

NexProv es la aplicación especializada en la gestión comercial y de productos.



3.1 Productos

Administración de los productos que la empresa maneja comercialmente.

Puede incluir:

- * Información del producto
- * Precios
- * Datos comerciales
- * Disponibilidad, cuando corresponda
- * Importación de información
- * Relaciones necesarias para cotizar

3.2 Gestión de productos

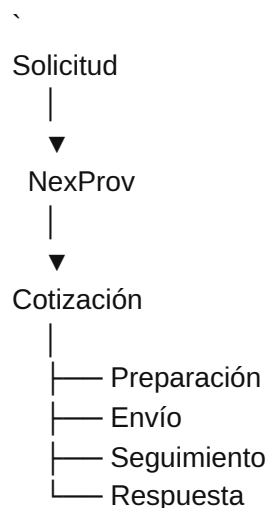
No se limita a mostrar productos.

Incluye las operaciones necesarias para mantener y utilizar la información de productos dentro de la actividad comercial.

3.3 Cotizaciones

NexProv tiene una participación principal en la parte comercial de las cotizaciones.

Una cotización puede surgir como respuesta a una solicitud.



Definiciones

Solicitud

Una empresa solicita productos o servicios.

Cotización

La empresa responde a esa solicitud ofreciendo productos o servicios bajo determinadas condiciones.

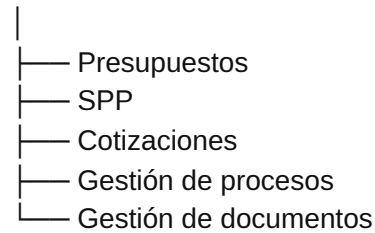
No se trata de un sistema para comparar cotizaciones de diferentes proveedores.

4\ GestiónPlus

GestiónPlus contiene la gestión empresarial y operativa que complementa la actividad comercial.

,

GESTIÓNPLUS



,

Entre sus responsabilidades están:

- * Presupuestos
- * SPP
- * Gestión de procesos
- * Documentación
- * Gestión relacionada con las cotizaciones

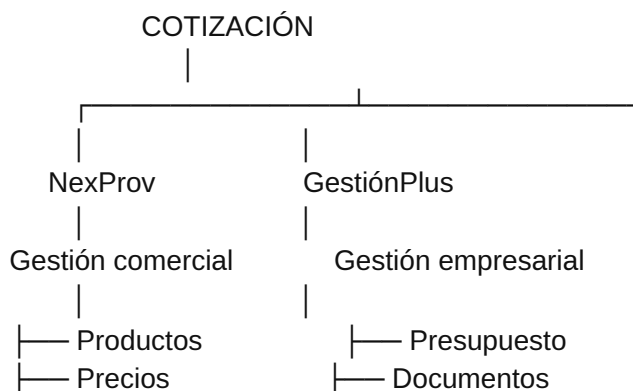
GestiónPlus también trabaja con las cotizaciones, pero desde una perspectiva diferente a NexProv.

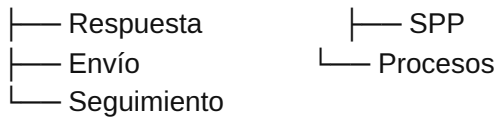
5\ Cotizaciones compartidas

La cotización debe ser una sola entidad dentro del backend.

No debe existir una cotización independiente en NexProv y otra diferente en GestiónPlus.

,





Ambas aplicaciones trabajan sobre la misma información.

Esto evita:

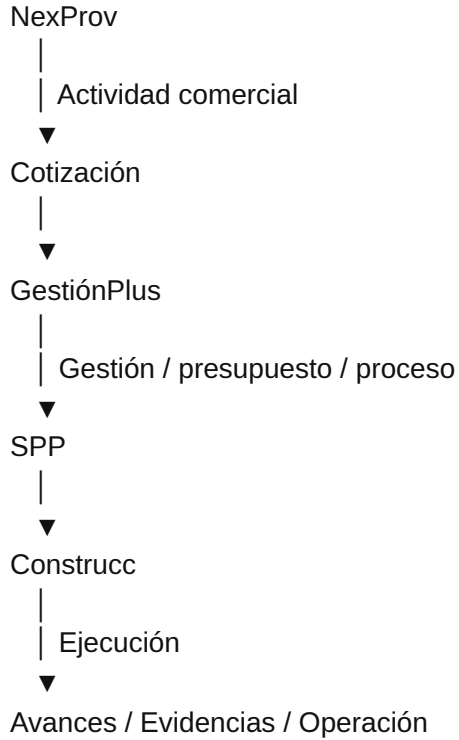
- * Duplicación de información
- * Inconsistencias
- * Cotizaciones diferentes para una misma operación
- * Problemas de sincronización
- * Lógica duplicada

6\ Construcc

Construcc es la aplicación especializada en la ejecución y operación.

Su función comienza cuando existe un proceso que debe ejecutarse y darle seguimiento operativo.

,



,

Responsabilidad de cada aplicación

NexProv
¿Qué voy a ofrecer?

- * Productos
- * Solicitudes
- * Cotizaciones
- * Gestión comercial

GestiónPlus

¿Cómo voy a gestionar lo que estoy haciendo?

- * Presupuestos
- * Cotizaciones
- * SPP
- * Documentos
- * Procesos
- * Gestión empresarial

Construcc

¿Cómo se está ejecutando?

- * Operación
- * Seguimiento
- * Avances
- * Evidencias
- * Ejecución del SPP

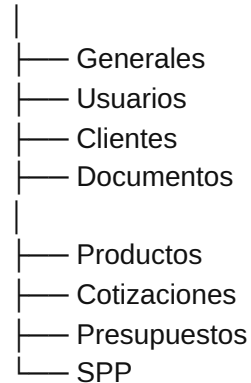
7\ La empresa como centro del ecosistema

El centro del sistema no es NexProv ni GestiónPlus.

El centro es la empresa.

,

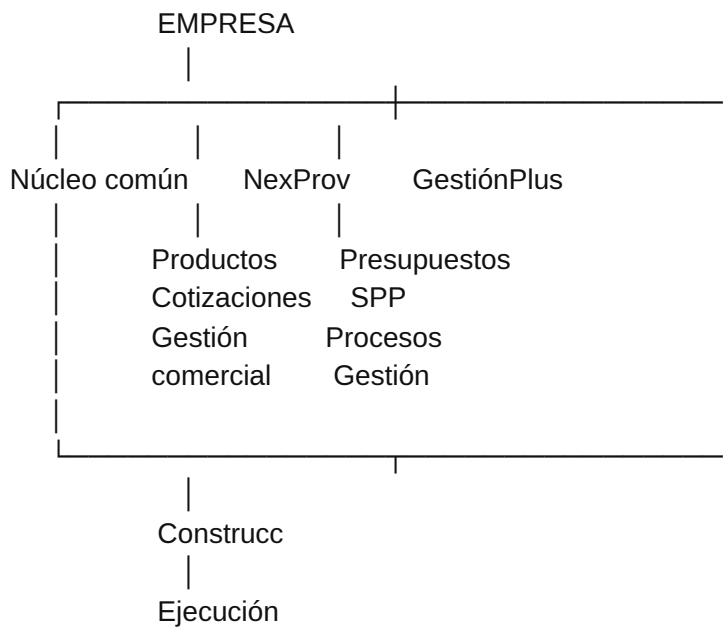
EMPRESA



,

Las aplicaciones representan diferentes vistas y responsabilidades sobre esa información.

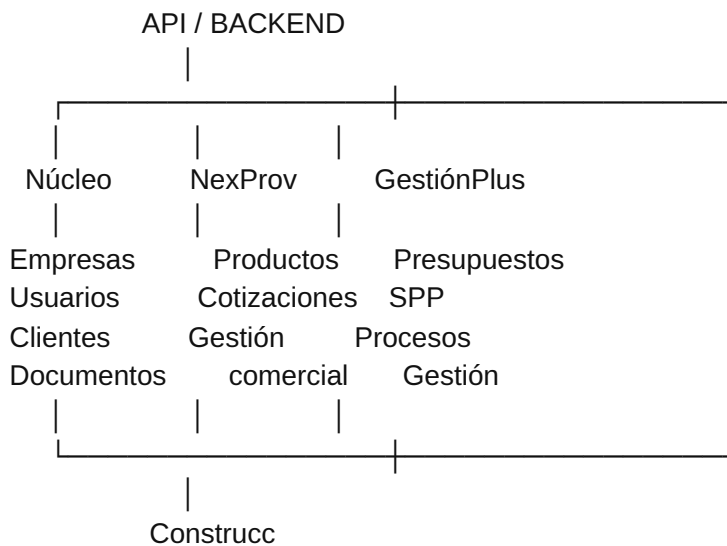
,



8\ Fuente de verdad

El backend/API compartido debe ser la fuente de verdad del ecosistema.

Las aplicaciones no deberían mantener copias independientes de las entidades principales.



Por ejemplo:

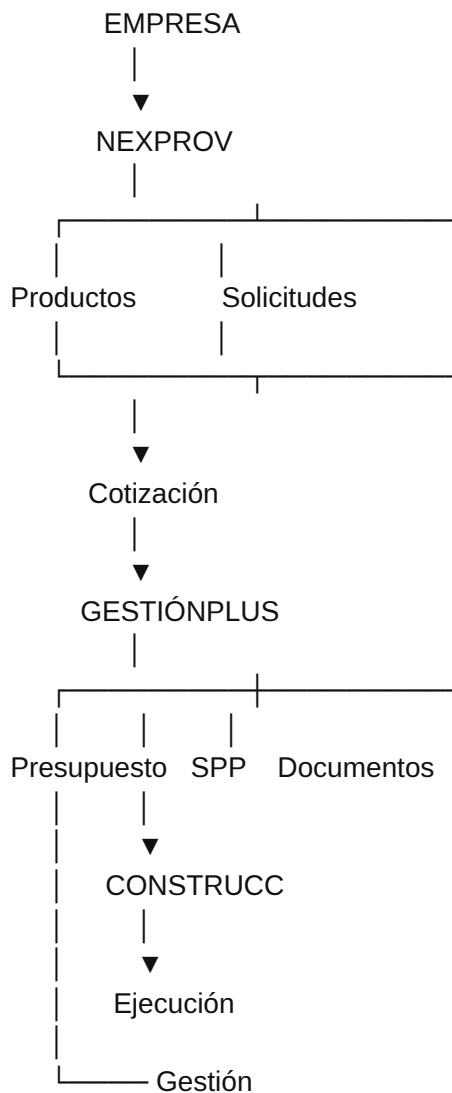
- * Un usuario debe existir una sola vez.
- * Un cliente debe existir una sola vez.
- * Un documento debe existir una sola vez.
- * Una cotización debe existir una sola vez.
- * Un producto debe tener una fuente de información definida.

* Las aplicaciones deben consumir y modificar la información mediante las reglas correspondientes de la API.

9\ Flujo general del ecosistema

El flujo conceptual puede representarse de la siguiente manera:

,



,

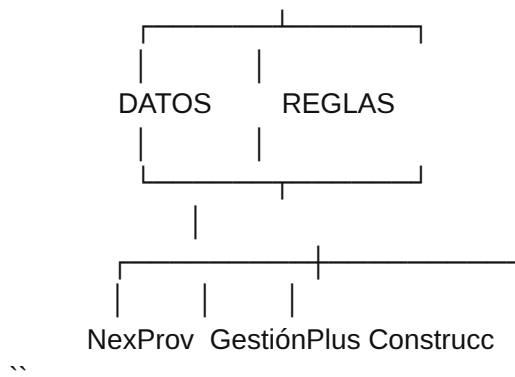
10\ Principio arquitectónico

La arquitectura debe seguir el principio:

Un núcleo empresarial común + aplicaciones especializadas.

,





Cada aplicación debe resolver una necesidad concreta sin convertirse en una copia de las demás.

Responsabilidades principales

Componente	Responsabilidad
Núcleo común	Información empresarial compartida
NexProv	Productos y gestión comercial
GestiónPlus	Gestión empresarial, presupuestos, SPP y procesos
Construcc	Ejecución y operación
API	Fuente de verdad y reglas del negocio
Gestor de documentos	Repositorio documental común

11\ Concepto final

El ecosistema puede resumirse así:

GestiónPlus proporciona el núcleo empresarial y la gestión; NexProv especializa la actividad comercial y los productos; Construcc especializa la ejecución operativa. Todos comparten una misma plataforma, API y fuente de información.

La intención no es construir tres sistemas aislados, sino una plataforma empresarial con diferentes aplicaciones especializadas según la función que desempeña cada usuario dentro del proceso.

Este documento ya está estructurado para usarlo como base de la definición técnica y funcional del proyecto.

Tú #41

Genera un Pdf bonito

ChatGPT #42

Data analysis isn't available right now. Do you want to continue without it?

If you want, I can:

- * Continuar con la generación del PDF del documento Markdown
- * Crear un resumen ejecutivo PDF del documento Markdown
- * Preparar un plan de implementación PDF basado en el documento

ChatGPT is AI and can make mistakes.